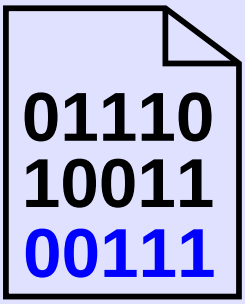
The dvdisaster Reed-Solomon Codec specification

The dvdisaster development team
support@dvdisaster.org



Version 1.02

Abstract

This is the *blue manual*, describing the data formats of the dvdisaster Reed-Solomon codecs which are currently called RS01, RS02, RS03 and RS04 (draft). The codecs create Reed-Solomon parity data to protect data stored on optical media. Depending on the codec, parity data can either be stored in a separate file or be integrated with the .iso image on the same medium. See `https://dvdisaster.jcea.es/`for additional resources on the dvdisaster project, e.g. for the *orange manual* (manual.pdf) containing information for end users.

Target audience. This paper is primarily intended as a working base for the dvdisaster developers and, when the final version has been crafted, as an implementation guide for third party developers who wish to create and process dvdisaster error correction data. It is **neither intended nor suitable** as end-user documentation; for usage information please refer to the online documentation at `https://dvdisaster.jcea.es/`.

Prerequisites. This paper assumes profound knowledge of coding theory and the underlying math. The reader is assumed to have a thorough understanding of Reed-Solomon codes, both in theory and from an implementation viewpoint. A basic understanding of programming in C is also assumed.

Copyright 2008-2015 Carsten Gnörlich.
Copyright 2021 The dvdisaster development team.
Verbatim copying and distribution of this entire article is permitted in any medium, provided this notice is preserved.

1 Changelog

Clarifications of specifications (without actual changes of implementations) are numbered with lowercase letters, e.g. *V1.00a*, *V1.00b* etc.

Changes which affect the implementation of codecs are indicated by increasing version numbers, e.g. *V1.00*, *V1.01*. Version numbering is independent for each codec.

1.1 RS04 codec

V1.00 *draft, experimental implementation in dvdisaster (command line)*
First draft of the specification: GF(2^{16}) code, layout, tag and CRC sector formats (appendix C), multi-session and audio rules, test vectors.
Clarified: the parity budget ρD is rounded half up; how dvdisaster applies the recognition steps; implementation limits.

1.2 RS03 codec

V1.00 *supported since dvdisaster version V0.79.4*
Clarified: RS03 header does not contain copy of first CRC sector (appendix A).
Added $sectorsPerLayer$ field in Ecc header and CRC block format.
Added ecc file specification.

1.3 RS02 codec

V1.01 *supported since dvdisaster version V0.79.5*
Added the *sectorsAddedByEcc* field in the *EccHeader*.

V1.00 *supported since dvdisaster version V0.66*
First draft of specification, open for review for missing parts and errors.

1.4 RS01 codec

V1.00 *supported since dvdisaster version V0.66*
First release of specification.

Contents

1 Changelog **2**
1.1 RS04 codec . . . 2
1.2 RS03 codec . . . 2
1.3 RS02 codec . . . 2
1.4 RS01 codec . . . 2

2 The RS04 codec **4**
2.1 Conventions . . . 4
2.2 Terminology . . . 5
2.3 Sector formats . . . 6
2.3.1 Sector to shard mapping . . . 6
2.4 The Reed-Solomon code . . . 7
2.4.1 Field and symbols . . . 7
2.4.2 Codewords . . . 7
2.4.3 Computing the code (informative) . . . 8
2.5 Layout of an RS04 set . . . 9
2.5.1 Codewords . . . 9
2.5.2 The added zone . . . 9
2.5.3 Augmented images and ecc files . . . 9
2.5.4 Example . . . 10
2.6 CRC sectors . . . 10
2.7 Tag sectors . . . 10
2.8 Checksums . . . 11
2.9 Choosing the encoding parameters (recommended) . . . 11
2.10 Multi-session discs . . . 13
2.11 Recognising RS04 data . . . 13
2.12 Audio CDs and read offsets . . . 14
2.13 Decoding (informative) . . . 15
2.14 Supplementary sets (informative) . . . 16
2.15 Reading media (informative) . . . 16
2.16 Versioning and compatibility . . . 17
2.17 Limits and edge cases . . . 18
2.18 Design rationale . . . 18
2.19 Test vectors . . . 19

3 The RS03 codec **20**
3.1 Physical layout . . . 20
3.2 Logical layout . . . 22
3.3 Calculating the layout for encoding . . . 22
3.4 Re-calculating the layout from defective media . . . 23
3.4.1 Using the Ecc Header . . . 23
3.4.2 Using the CRC layer . . . 24
3.4.3 Evaluating the Reed-Solomon code . . . 24
3.5 Contents of the CRC layer . . . 25

3.6 Encoding the ecc layers . . . 25
3.7 Encoding as a separate error correction file . . . 25

4 The RS02 codec 27
4.1 Physical layout . . . 27
4.2 Logical layout . . . 29
4.3 Calculating the image layout for encoding . . . 30
4.3.1 Automatic layout calculation . . . 30
4.3.2 Layout calculation by user selected criteria . . . 32
4.3.3 Layout calculation from ecc header information . . . 32
4.4 Automatic layout calculation example . . . 33
4.5 Re-calculating the layout from defective media . . . 33
4.6 Sector addressing and initialization scheme . . . 34
4.7 Encoding the checksums . . . 37
4.8 Encoding the ecc layers . . . 38

5 The RS01 codec 40
5.1 Physical layout . . . 41
5.2 Logical ecc file layout . . . 42
5.3 Calculating the layout for encoding . . . 43
5.4 Getting the layout when recovering defective media . . . 43
5.5 md5 checksums . . . 43
5.6 Special cases . . . 43

A The common Ecc header format 44

B RS03 CRC block format 46

C RS04 tag and CRC sector format 48

D RS(255,k) encoding parameters and examples 50

2 The RS04 codec

This section describes the data format of the dvdisaster RS04 codec. **Status: draft.** dvdisaster implements RS04 as an experimental codec (command line only); the format may still change until this section is declared final (see the changelog).

RS04 keeps the basic idea of RS03 – the medium is divided into layers and error correction block ("codeword") i consists of sector i of every layer – but changes almost everything else:

- The Reed-Solomon code works over GF(2^{16}) instead of GF(2^8). A codeword has up to 65535 symbols instead of 255, so the number of layers n can be much larger and each codeword is spread over many more places on the medium. Against scattered damage this brings the tolerated loss close to the theoretical limit, while damage at the outer edge is handled as well as by RS03 (section 2.18).

- The layout is *self-describing*. Small *tag sectors* are replicated throughout the error correction data. Each of them describes the complete layout, so a single readable tag is enough. No table of medium sizes and no brute-force search are needed, and the augmented image may have any size.

- Several RS04 sets may exist on one medium (one per session of a multi-session disc). All positions are relative to the start of the set.

- Besides 2048-byte sectors, Mode 2 Form 2 sectors (2324 bytes) and CD-DA audio sectors (2352 bytes) can be protected. Audio extraction with a different drive read offset is handled by an offset search.

- The error correction file ("ecc file") is byte-for-byte the same data that is appended to the image in the augmented case.

Everything in this section is normative unless it is marked as *recommended* or *informative*. Recommended procedures describe what dvdisaster does; other encoders may choose differently, and decoders must accept every set which satisfies the validity rules.

2.1 Conventions

- A *sector* is a block of B bytes, where B is the sector size of the set (section 2.3). Positions and counts are given in sectors and start at 0.

- All integers on disc are stored *little-endian*, independent of the host. Big-endian hosts must convert them after reading and before writing (as dvdisaster does for RS03 with `SwapEccHeaderBytes()`).

- $\lceil x \rceil$ and $\lfloor x \rfloor$ round up and down. $\mathrm{NextPow2}(x)$ is the smallest power of two $\geq x$ ($\mathrm{NextPow2}(1) = 1$), $\mathrm{Pow2Floor}(x)$ the largest power of two $\leq x$ for $x \geq 1$, and $\mathrm{Pow2Floor}(0) = 1$.

- *Crc32* is the function `Crc32()` of dvdisaster which is also used by RS01–RS03: reflected polynomial `0xEDB88320`, initial value `0xFFFFFFFF`, *no* final inversion. It equals the zlib `crc32()` value XOR `0xFFFFFFFF`. Check values: Crc32("123456789") = `0x340BC6D9`; 2048 zero bytes give `0x0E174561`, 2324 zero bytes `0x2F6ED567` and 2352 zero bytes `0x416831C0`. Crc32 values are stored as little-endian 32-bit integers.

- *MD5* is the message digest of RFC 1321.

2.2 Terminology

An *RS04 set* consists of the protected *data sectors* and the *added zone* which RS04 creates. The added zone holds tag sectors, CRC sectors and parity sectors. The following symbols are used throughout this section:

B	sector size: 2048, 2324 or 2352 bytes
B_s	shard size $64\lceil B/64 \rceil$: 2048 or 2368 bytes
S	number of data sectors, $S = \lceil dataBytes/B \rceil \geq 1$
c	number of CRC sectors, $c = \lceil S/512 \rceil$
D	number of protected sectors, $D = S + c$
L	layer size (sectors per layer) ≥ 1
k	number of data layers = original symbols per codeword; $kL \geq D$
m	number of ecc layers = parity symbols per codeword; $1 \leq m \leq k$
n	codeword length $n = k + m$
M	$\mathrm{NextPow2}(m)$; the code requires $M + k \leq 65536$
T	tag interval: one tag sector every T sectors of the added zone
Q	number of CRC and parity sectors, $Q = c + mL$
A	size of the added zone, $A = Q + \lceil Q/(T-1) \rceil$

The stored parameters are B, $dataBytes$, L, k, m and T; all other values are derived from them.

```
relative position 0                    S                              S+A
|<--------- data sectors -------->|<------- added zone ------->|
| D_0 D_1 ...               D_S-1 | t C..C P..P t P..P t P..P  |
   t = tag (every T sectors), C = CRC sector, P = parity sector

logical view       column 0    column 1    ...  column L-1
  data layer 0     X_0         X_1              X_L-1
  data layer 1     X_L         X_L+1            X_2L-1
  ...
  data layer k-1   X_(k-1)L    ...              X_kL-1
  ecc layer 0      P_0         P_1              P_L-1
  ...
  ecc layer m-1    P_(m-1)L    ...              P_mL-1

X_t = D_t (t < S), C_(t-S) (S <= t < D), zero sector (D <= t < kL)
codeword i = column i
```

2.3 Sector formats

B	Contents of a data sector	B_s	tail r
2048	user data of Mode 1 and Mode 2 Form 1 CD sectors, DVD and BD sectors (ISO and UDF images)	2048	0
2324	user data of Mode 2 Form 2 CD sectors (e.g. VCD/SVCD video)	2368	20
2352	CD-DA audio: 588 stereo samples of 4 bytes each (16-bit signed little-endian left channel, then right channel), as returned by READ CD and stored in `.bin` rips	2368	48

All sectors of a set – data, tag, CRC and parity sectors – have exactly B bytes. Augmented images and ecc files store them back-to-back without any per-sector framing (no sync pattern, header, subheader or EDC/ECC of the physical CD sector). The sector size is stored in every tag.

Last data sector. The data is $dataBytes$ bytes long. If $dataBytes$ is not a multiple of B, the last data sector is padded with zero bytes to B bytes for encoding and for its Crc32. The padding is not part of the data; it is not covered by the MD5 sum. Augmented images require $dataBytes = S \cdot B$, otherwise the added zone would not start on a sector boundary. Ecc files may protect data of any length, e.g. arbitrary files.

Missing sectors in image files. dvdisaster stores a sector which could not be read as its *dead sector marker*: 2048 bytes starting with the text `dvdisaster dead sector marker` (the marker used for RS01–RS03 images), or 2048 copies of a fill byte if the user requested one (e.g. zero for images made by other tools). For $B > 2048$ a missing sector is this 2048-byte marker followed by $B - 2048$ zero bytes (fill bytes if a fill byte is used). Readers recognise a missing sector by its first 2048 bytes only. A missing sector is an erasure; its Crc32 is meaningless.

Whole-disc images (informative). dvdisaster reads a multi-session medium as one image in which every sector is at its absolute position. The sectors between the sessions (lead-out, lead-in and pregap) can not be read; they are stored as zero-filled B-byte sectors. They belong to no set (section 2.10) and are not erasures.

2.3.1 Sector to shard mapping

The Reed-Solomon code (section 2.4) works on *shards* of B_s bytes in the buffer format of Leopard-RS: the shard is a sequence of 64-byte blocks, and byte u ($0 \le u < 32$) of block b is the low byte, byte $u + 32$ the high byte of the 16-bit symbol $32b + u$. A sector is turned into a shard as follows, with $F = 64\lfloor B/64 \rfloor$ and $r = B - F$:

- shard bytes $0 \ldots F - 1$ = sector bytes $0 \ldots F - 1$;
- shard bytes $F \ldots F + r/2 - 1$ = sector bytes $F \ldots F + r/2 - 1$ (low bytes of the first $r/2$ symbols of the last block);

- shard bytes $F+32 \ldots F+32+r/2-1$ = sector bytes $F+r/2 \ldots B-1$ (high bytes of the same symbols);
- all other shard bytes are zero.

For $B = 2048$ the shard is the sector itself. For a parity shard the bytes which are zero in this mapping are always zero: the symbols they belong to are zero in every original shard and the code is linear. Therefore a parity shard is turned back into a B-byte parity sector by the inverse mapping, and no padding is ever stored. Decoders may check that the unused shard bytes of decoded shards are zero.
Note: simply appending zero bytes to the sector would not work for $B \neq 2048$, since the parity would then have non-zero high bytes for the partially used symbols of the last block (20 resp. 24 extra bytes) which do not fit into a B-byte parity sector.

2.4 The Reed-Solomon code

2.4.1 Field and symbols

RS04 computes in GF(2^{16}) = GF(2)$[z]/(p(z))$ with $p(z) = z^{16}+z^5+z^3+z^2+1$ (`0x1002D`). Writing a field element as a 16-bit integer whose bit b is the coefficient of z^b gives its *polynomial representation*. The symbols stored on disc use a different representation, based on the following *Cantor basis* $\beta_0, \ldots, \beta_{15}$ (given in polynomial representation):

β_0	β_1	β_2	β_3	β_4	β_5	β_6	β_7
`0001`	`ACCA`	`3C0E`	`163E`	`C582`	`ED2E`	`914C`	`4012`
β_8	β_9	β_{10}	β_{11}	β_{12}	β_{13}	β_{14}	β_{15}
`6C98`	`10D8`	`6A72`	`B900`	`FDB8`	`FB34`	`FF38`	`991E`

The basis satisfies $\beta_0 = 1$ and $\beta_i^2 + \beta_i = \beta_{i-1}$ for $1 \leq i \leq 15$, which is a useful check for an implementation. A stored 16-bit symbol value $x = \sum_b x_b 2^b$ denotes the field element

$$\langle x \rangle = \sum_{b=0}^{15} x_b \, \beta_b .$$

Addition of symbols is therefore a bitwise XOR of their values, while multiplication has to be carried out on $\langle\cdot\rangle$. Leopard-RS does this with logarithm tables taken with respect to the generator z (`0x0002` in polynomial representation), indexed directly by the stored symbol value. Examples (right hand sides in polynomial representation): $\langle 1 \rangle =$ `0001`, $\langle 2 \rangle =$ `ACCA`, $\langle 3 \rangle =$ `ACCB`, $\langle 4 \rangle =$ `3C0E`, and $\langle 2 \rangle \cdot \langle 3 \rangle = \langle 1 \rangle$.

2.4.2 Codewords

Let $k \geq 1$, $1 \leq m \leq k$, $M = \mathrm{NextPow2}(m)$ and $M + k \leq 65536$. The evaluation points are $\omega_j = \langle j \rangle$ for $0 \leq j < M + k$; they are pairwise different. An RS04 codeword with the original symbols $d_0, \ldots, d_{k-1}$ is the unique vector $(c_0, \ldots, c_{M+k-1})$ of symbols with

$$c_{M+i} = d_i \quad (0 \leq i < k) \qquad \text{and} \qquad \sum_{j=0}^{M+k-1} \langle c_j \rangle \, \omega_j^t = 0 \quad (0 \leq t < M). \qquad (1)$$

For given d_i, (1) is a linear system for $c_0, \ldots, c_{M-1}$ whose matrix (ω_j^t) is a Vandermonde matrix of distinct points, so the solution exists and is unique. The *parity symbols* are $c_0, \ldots, c_{m-1}$; c_j is stored in ecc layer j. The symbols $c_m, \ldots, c_{M-1}$ are never stored.

The equations (1) define a generalized Reed-Solomon code of length $M + k$ and dimension k (the dual of the code of polynomials of degree $< M$ evaluated at ω_j). It is MDS, and so is the stored code of length $n = k + m$ obtained by omitting $c_m, \ldots, c_{M-1}$: **any k of the n stored symbols determine the codeword**, i.e. up to m erasures per codeword can be corrected.

Special cases. For $m = 1$ ($M = 1$), (1) reduces to $c_0 = d_0 \oplus d_1 \oplus \cdots \oplus d_{k-1}$ (XOR parity). For $k = 1$, m must be 1 and the parity is a copy of the original. Both cases need no special treatment; they follow from (1).

Limits. The limit is $M + k \leq 65536$ and *not* $n \leq 65536$: $k = 49152, m = 16384$ is valid, but $k = 49153, m = 16383$ is not. $k \leq 65535$ follows.

Shards. The code is applied to shards symbol by symbol: for every symbol index s ($0 \leq s < B_s/2$), symbol s of the k original shards and of the m parity shards of a codeword satisfy (1).

2.4.3 Computing the code (informative)

The code (1) is exactly the GF(2^{16}) code of Leopard-RS by Christopher A. Taylor (`https://github.com/catid/leopard`, file `LeopardFF16.cpp` of the commit of 2024-03-23

`6e5725ebdf9da4370b0bcc4f70fa8eb66f4e6198`

which is the reference for bit-exactness). Leopard computes it with the additive FFT over the novel polynomial basis of Lin, Chung and Han:

- S.-J. Lin, W.-H. Chung, Y. S. Han: *Novel polynomial basis and its application to Reed-Solomon erasure codes*, FOCS 2014.
- S.-J. Lin, T. Y. Al-Naffouri, Y. S. Han, W.-H. Chung: *Novel polynomial basis with fast Fourier transform and its application to Reed-Solomon erasure codes*, IEEE Trans. Inf. Theory 62(11), 2016.

The encoder takes the originals in chunks of M, computes the inverse FFT of each chunk with skew offset $M(i + 1)$ for chunk i (zero-padding the last chunk), XORs the results and applies a forward FFT of size M; output j is c_j. The decoder evaluates an error locator with a Walsh-Hadamard transform modulo 65535 and needs work space for $N = \mathrm{NextPow2}(M + k)$ shards (the *FFT size*). A skew factor logarithm of 65535 means "multiply by zero" (skip). The log/exp tables, skew factors and Walsh-Hadamard table are built exactly as in `LeopardFF16.cpp`.

Usable implementations are `src/rs04-ff16.c` (a C port of the FF16 code path), Leopard-RS built with GF(2^{16}) only, and the high-rate engine of the Rust crate `reed-solomon-simd`. **Caution:** the stock `leo_encode()` switches to GF(2^8) when $\mathrm{NextPow2}(M + k) \leq 256$ and then produces different parity (except for $m = 1$ and $k = 1$); RS04 always uses GF(2^{16}). Any other method, e.g. solving (1) directly, gives the same result. `contrib/rs04-reference.py` is a slow reference which does exactly that.

Example. For a single symbol column with $k = 3$, $m = 2$ and originals `0001 0002 0003`, the parity symbols are `000C 000C`. With $k = 5$, $m = 3$ ($M = 4$) and originals `0001`...`0005` they are `000E 0001 000C`. With $m = 1$ the parity of `0001 0002 0003` is `0000`.

2.5 Layout of an RS04 set

2.5.1 Codewords

The *original sectors* $X_0, \ldots, X_{kL-1}$ are the data sectors, followed by the CRC sectors, followed by zero sectors:

$$X_t = \begin{cases} D_t & 0 \le t < S \\ C_{t-S} & S \le t < D \\ \text{all-zero sector} & D \le t < kL \end{cases}$$

The zero sectors are virtual: they are never stored and never read, and decoders treat them as known (not as erasures). $kL - D$ may exceed $L - 1$, i.e. complete *padding layers* are allowed; they make redundancies above 100% possible in spite of $m \le k$.

Codeword i ($0 \le i < L$) has the originals $d_j = X_{jL+i}$ ($0 \le j < k$) and the parity sectors $\Pi_{jL+i} = c_j$ ($0 \le j < m$), computed from the shards of these sectors as described in sections 2.3.1 and 2.4. Thus data layer j consists of the sectors $jL \ldots jL + L - 1$ of the data, and ecc layer j of the parity sectors $\Pi_{jL} \ldots \Pi_{jL+L-1}$. This is the RS03 rule with a larger number of layers; there is no permutation inside layers.

2.5.2 The added zone

The added zone stores the $Q = c + mL$ *payload sectors* $C_0, \ldots, C_{c-1}, \Pi_0, \ldots, \Pi_{mL-1}$ (in this order, payload index $q = 0 \ldots Q - 1$) and the tag sectors. Sector a of the added zone ($0 \le a < A$) is

- tag sector number $g = a/T$ if $a \bmod T = 0$;
- otherwise payload sector $q = a - \lfloor a/T \rfloor - 1$.

Conversely, payload sector q is stored at $a = q + \lfloor q/(T-1) \rfloor + 1$. There are $\lceil Q/(T-1) \rceil$ tags and $A = Q + \lceil Q/(T-1) \rceil$; the last group may be shorter than T sectors, but every tag is followed by at least one payload sector. Sector a of the added zone has the *relative position* $S + a$.

Consequences: the first sector of the added zone, directly behind the data, is a tag; any T consecutive sectors of the added zone contain a tag; the last T sectors of the added zone contain a tag.

2.5.3 Augmented images and ecc files

An **augmented image** consists of the S data sectors followed by the A sectors of the added zone; it is $S + A$ sectors long. The data sectors are unchanged, so the image remains usable (e.g. mountable).

An **ecc file** is the added zone alone: $A \cdot B$ bytes, sector a at byte offset $a \cdot B$. Its contents are identical to the added zone of the augmented image, so concatenating the data and the ecc file yields the augmented image and vice versa. There is no separate ecc header; the first sector of the file is a tag. A tag begins with the same 16 bytes `*dvdisaster*RS04` that an `EccHeader` with method "RS04" would have, so programs which look for the ecc header cookie at the start of a file recognise the method. All positions stored in tags are relative to the set start and therefore the same in both cases.

2.5.4 Example

The format test vector F1 (section 2.19) has $B = 2048$, $dataBytes = 81820$ (so $S = 40$, the last sector holds 1948 bytes), $c = 1$, $D = 41$, $L = 7$, $k = 6$, $m = 3$ and $T = 8$. The originals are $X_0 \ldots X_{39} = D_0 \ldots D_{39}$, $X_{40} = C_0$ and the zero sector X_{41}. $Q = 22$, there are 4 tags and $A = 26$:

added zone sector a (relative position $40 + a$)	contents
0	tag 0 (covers $C_0, \Pi_0 \ldots \Pi_5$)
1	C_0
2 ... 7	$\Pi_0 \ldots \Pi_5$
8	tag 1 (covers $\Pi_6 \ldots \Pi_{12}$)
9 ... 15	$\Pi_6 \ldots \Pi_{12}$
16	tag 2 (covers $\Pi_{13} \ldots \Pi_{19}$)
17 ... 23	$\Pi_{13} \ldots \Pi_{19}$
24	tag 3 (covers Π_{20})
25	Π_{20}

Codeword 3 consists of the data sectors 3, 10, 17, 24, 31, 38 and the parity sectors $\Pi_3, \Pi_{10}, \Pi_{17}$ at the relative positions 45, 53 and 61. Codeword 5 contains $X_{40} = C_0$, and codeword 6 contains the zero sector X_{41} as its last original.

2.6 CRC sectors

CRC sector C_j ($0 \le j < c$) holds 512 little-endian 32-bit values; value u (bytes $4u \ldots 4u + 3$) is the Crc32 of data sector D_{512j+u} if $512j + u < S$ and zero otherwise. The Crc32 is taken over the B bytes of the sector as it is encoded, i.e. including the zero padding of the last data sector. Bytes $2048 \ldots B - 1$ of a CRC sector are zero.

CRC sectors are originals of the code, so lost CRC sectors are recovered by the error correction like data sectors (as with the RS03 CRC layer). They have no header because every tag describes the layout, and their integrity is protected by the Crc32 values in the tags.

2.7 Tag sectors

The byte layout of a tag is given in appendix C. A tag holds the complete layout of the set and the Crc32 of the payload sectors that follow it: tag g covers the payload sectors $g(T-1) \le q < \min(Q, (g+1)(T-1))$ and stores the Crc32 of payload sector q in $crc[q - g(T-1)]$; unused

entries are zero. Tags are not part of the code. They are protected by replication instead: all tags of a set are identical except for $ownPosition$, $selfCRC$ and $crc[\,]$.

A sector is a **valid tag** if all of the following holds; otherwise it is not a tag (it may be a data or parity sector which looks similar by chance):

1. bytes 0–15 are `*dvdisaster*RS04`;
2. $selfCRC$ equals the Crc32 of bytes 0–2047 of the sector with bytes 28–31 set to zero;
3. $sectorBytes \in \{2048, 2324, 2352\}$ and $tagInterval$ is a power of two with $2 \leq T \leq 256$;
4. $dataBytes \geq 1$, $L \geq 1$, $1 \leq m \leq k$, $\mathrm{NextPow2}(m) + k \leq 65536$ and $kL \geq D$;
5. $ownPosition \geq S$, $(ownPosition - S) \bmod T = 0$ and $ownPosition - S < A$.

If 1 and 2 hold but $version$ or $layoutVariant$ is not supported, the tag belongs to a set of a newer format; see section 2.16. Readers should use the values held by the majority of the valid tags of a set if tags disagree.

2.8 Checksums

Checksum	Stored in	Covers	Purpose
Crc32 per data sector	CRC sectors	the B bytes of D_t (last sector zero-padded)	detect damaged data sectors (erasures); audio read offset detection
Crc32 per payload sector	tags	the B bytes of each CRC and parity sector	detect damaged CRC and parity sectors (erasures)
$selfCRC$	each tag	bytes 0–2047 of the tag with $selfCRC = 0$	recognise valid tags
$dataMD5$	each tag	exactly the $dataBytes$ bytes of data, without padding	end-to-end check after repair; matching image and ecc file

Zero sectors and the parity symbols $c_m \ldots c_{M-1}$ are not stored and have no checksum. Unlike RS01/RS02 there is no MD5 sum over the error correction data: the per-sector Crc32 values in the tags check every CRC and parity sector individually and also tell *which* sector is damaged, which a single MD5 sum cannot.

2.9 Choosing the encoding parameters (recommended)

Any parameters which satisfy the validity rules of section 2.7 can be decoded. dvdisaster chooses them as follows.

1. Parity budget E.

- *Ecc files* and explicitly requested redundancy: $E = \lfloor \rho D + 1/2 \rfloor$ (rounded half up) for the requested redundancy ρ (parity sectors per protected sector). The size of the set is $C_{tot} = D + E$. dvdisaster takes ρ from the option `-n`: p`%` gives $\rho = p/100$, r`r` ("roots" as for RS03; also without the suffix) $\rho = r/(255 - r)$, `normal` (the default) $32/223$ and `high` $64/191$. x`m` chooses the largest ρ for which the ecc file has at most x MiB.

- *Augmented images (default): fill the medium.* Let C be the number of sectors available for the whole set: the capacity of the target medium (as reported by the drive, from the medium size table also used by RS02/RS03, or given by the user), or for a further session of a multi-session disc the free space of that session. With $Z = C - S$ and $T_0 = \mathrm{TagInterval}(Z)$ the largest number of payload sectors that fits is $Q_{max} = Z - \lceil Z/T_0 \rceil$, and $E = Q_{max} - c$. $C_{tot} = C$.

- In both cases E is capped at $2D$ (200% redundancy, as RS03). Encoding is refused if $E < 1$ or $E/(D + E) < 8/255$ (less than the 8 roots RS03 requires as a minimum); a warning is given if $E < D/5$ (less than 20% redundancy).

2. FFT size limit N.

$$N = \min\left(65536,\ \mathrm{Pow2Floor}\left(\left\lfloor \frac{C_{tot}}{32} \right\rfloor\right),\ \mathrm{Pow2Floor}\left(\left\lfloor \frac{R}{128 \cdot B_s} \right\rfloor\right)\right)$$

where R is the memory budget of the repair (default 4 GiB, dvdisaster option `--rs04-memory`). A repair pass holds 128 consecutive codewords of up to N shards each in memory (with fewer codewords per pass a hard disk seeks too much); with 4 GiB this gives $N = 16384$ for DVD and BD. The $C_{tot}/32$ term keeps $L \geq 32$ on small media ($N = 8192$ for CD). The user may override N. If no geometry satisfies step 3 with this N (only for very small sets), $N = 65536$ is used.

3. Geometry. L is the smallest integer $L \geq \min(32, D, E)$ for which

$$m = \lfloor E/L \rfloor \geq 1, \quad k = \max(\lceil D/L \rceil, m) \quad \text{and} \quad \mathrm{NextPow2}(m) + k \leq N.$$

The left side of the last condition does not increase with L, so L can be found by bisection. If no $L \leq E$ qualifies, encoding is refused. $k > \lceil D/L \rceil$ (padding layers) only happens for redundancies above 100%. The smallest L gives the longest codewords for the memory budget, and at most $L - 1$ sectors of the budget remain unused.

4. Tag interval. $T = \mathrm{TagInterval}(Q)$ with $Q = c + mL$, where

$$\mathrm{TagInterval}(x) = \begin{cases} 256 & x \geq 4080 \\ \max(2, \mathrm{Pow2Floor}(\lfloor x/16 \rfloor)) & \text{otherwise} \end{cases}$$

This gives every set at least 16 tags (or Q tags if $Q < 32$); large sets spend $1/256$ of the added zone on tags. When filling a medium, $T = T_0$ is used instead if the set would not fit with T.

5. Set ID. 8 random bytes, newly chosen for every encoding run.

Rationale for $L \geq 32$: DVD drives lose data in units of 16 sectors (one ECC block) and BD drives in units of 32 sectors (one cluster). With $L \geq 32$ such a unit costs each codeword at most one symbol.

Examples (produced by `contrib/rs04-reference.py`; "fill" uses the medium sizes of table 1):

Case	S	L	k	m	T	FFT size	set size	parity/D
CD, fill	300000	49	6135	1196	256	8192	359423	19.5%
DVD, fill	2000000	142	14113	2042	256	16384	2295024	14.5%
DVD-9, fill	3500000	286	12262	2315	256	16384	4171550	18.9%
BD-25, fill	10000000	816	12279	2205	256	16384	11825945	18.0%
CD, fill, tiny image	100	32	6	6	8	16	321	190%
ecc file, $\rho = 32/223$	2000000	141	14213	2039	256	16384	2292549	14.3%
CD-DA ecc file, 20%	330000	54	6124	1224	256	8192	397003	20.0%
ecc file, 50%, 10 sectors	10	6	2	1	2	4	24	54.5%

2.10 Multi-session discs

Each session of a multi-session disc may carry its own RS04 set, which protects the sectors of that session only. The set is created from the session image (as written by the burning software, e.g. the image created for `growisofs -M`) without knowing its absolute position on the disc: all positions in the set are relative to its first data sector. When the whole disc is read, the absolute start of a set is

$$start = (\text{absolute LBA of a tag}) - ownPosition,$$

and the set occupies the absolute sectors $start \ldots start + S + A - 1$. Gaps between sessions (lead-out, lead-in) belong to no set. An ISO 9660 file system in a later session may refer to files of earlier sessions; those sectors are protected by the set of the session they are stored in.

CD Extra (an audio session followed by a data session) is covered by the same rules: the audio session is protected by an ecc file with $B = 2352$ (section 2.12), which may be stored as a file in the data session, and the data session may carry its own augmented set.

2.11 Recognising RS04 data

Augmented images and media. A reader looks for valid tags in the following order and stops as soon as it has found what it needs:

1. If the image (or, on a multi-session medium, a session) starts with an ISO 9660 file system of V sectors, the sectors V and $V + 150$ relative to its start (behind the file system, and behind the 150 sectors of padding added by some mastering programs). For an image which was not padded after the file system, sector V is tag 0.

2. The last 256 sectors of the image or session. They contain a tag if the image ends with the complete added zone (since $T \leq 256$).

3. All sectors, from the end towards the start (the added zone is at the end of a set).

Every valid tag yields a *candidate set* $(setID, start)$ with $start = position - ownPosition$; candidates with $start < 0$ are discarded. Tags with equal $setID$ and $start$ belong to the same set. A set may extend beyond the end of the image (a truncated image is repaired by recovering the missing sectors). The size of the complete image is $start + S + A$; dvdisaster uses it as the expected image size when reading a medium.

To enumerate all sets of a multi-session disc, the reader applies steps 1 and 2 to every session in the table of contents, or step 3 to a disc image without table of contents, and reports every candidate set.

Stale and embedded sets. Candidates may also come from tags of an earlier recording on rewritable media, or from an ecc file or augmented image which is stored as a *file* inside the protected data. A reader confirms a candidate by comparing up to 16 readable data sectors with their Crc32 values from readable CRC sectors. A candidate whose compared sectors all mismatch is rejected. If confirmed candidates overlap, the one with more matching sectors wins, then the one with more valid tags. A candidate without readable data or CRC sectors is kept but reported as unconfirmed.
Informative: dvdisaster applies steps 1 and 2 to every image it opens. Step 3 is applied to the parts of an image file which are not covered by the sets found so far (gaps between and before sessions), and to a whole image without any set found only on request (option `-a RS04`), since reading every sector of every opened image would be too slow. A set whose tag 0 is lost and whose image has more than 256 sectors appended is therefore only found with `-a RS04`. On a medium, steps 1 and 2 are used, step 2 also for the end of every session in the table of contents. To confirm a candidate, dvdisaster tries up to 16 CRC sectors spread over the set and the first 16 data sectors covered by each of them.

Ecc files. Sector 0 of an ecc file is tag 0. If it is unreadable or invalid, the reader searches the file for the 16-byte signature at byte offsets which are multiples of 4 (all three sector sizes are multiples of 4) and accepts a valid tag at offset o only if $o = (ownPosition - S) \cdot sectorBytes$. The ecc file must be $A \cdot B$ bytes long; shorter files are treated as truncated (the missing sectors are erasures). An ecc file holds one set; if valid tags at their proper places belong to different sets (other $setID$) or disagree, the set held by the majority of the tags is used, and tags of other sets count as invalid.
Informative: dvdisaster takes tag 0 and the tags in the last 256 sectors of the file. If they describe different sets, it compares up to 16 tag positions of each set, spread over the file, and uses the set with the most valid tags at them (tag 0 wins a tie). This catches e.g. a tag 0 which belongs to another ecc file after damage to the file system.

Recognition never depends on the medium size, the medium type or a table of known sizes.

2.12 Audio CDs and read offsets

Audio CDs are protected by ecc files made from a rip which covers a fixed range of the disc (normally all tracks as one `.bin` file, $B = 2352$). CD drives differ in their *read offset*: two rips of the same disc made with different drives (or different offset corrections) are shifted by a

whole number of samples (4 bytes) against each other. RS04 finds the shift with the data sector Crc32 values.

Let R be the new rip (a byte string) and crc_t the stored Crc32 of data sector t. For an offset δ (in samples, may be negative) the *aligned sector* t is the 2352 bytes of R starting at byte $2352\,t+4\delta$. The recommended procedure is:

1. **Probe sectors.** Take every data sector t with $10 \le t < S - 10$ whose crc_t is known from a readable CRC sector (a lost CRC sector can only be recovered after the rip has been aligned, since its codeword contains data sectors) and differs from `0x416831C0`, the Crc32 of digital silence (silence matches at every offset). Choose up to 64 of them, evenly spread over this range.

2. **Fast path.** If all probe sectors match for $\delta = 0$, the rip is aligned.

3. **Search.** For every δ with $|\delta| \le 5880$ samples (10 sectors; drive read offsets are usually below 2000 samples in magnitude, and two rips differ by the difference of their offsets), the score is the number of probe sectors whose aligned sector has the Crc32 crc_t. Probe sectors whose aligned sector is not completely inside R or touches a sector which was unreadable in the new rip do not count.

4. **Decision.** The offset with the highest score is accepted if the score is at least 2 and at least twice the second highest score. Otherwise (e.g. too much damage, or the audio repeats exactly with a period of whole samples) the user has to specify the offset (dvdisaster option `--audio-offset`); manual offsets are not limited to the search window.

With the accepted δ, aligned sectors which are not completely inside R or overlap unreadable parts of the rip are erasures. With $\delta \neq 0$ this concerns the first or the last $\lceil 4|\delta|/2352 \rceil$ sectors, because drives normally cannot read into the lead-in or lead-out; they are recovered like any other erasure, and the repaired rip is identical to the original one ($dataMD5$ matches). dvdisaster repairs such a rip by first rewriting the file shifted by 4δ bytes (missing bytes at the edges become zero) and then decoding it with $\delta = 0$. Rips which differ in more than a constant shift (other track splitting, pregaps or hidden track audio removed) are out of scope; the user has to provide a rip of the same range. Reading audio sectors from a drive (READ CD with C2 error pointers) does not affect the format.

2.13 Decoding (informative)

1. Recognise the set and read its tags (ignoring tags with another $setID$), CRC sectors and parity sectors. An added zone sector is an erasure if it is unreadable or its Crc32 does not match the value in its tag. If its tag is lost, a readable sector is assumed to be correct.

2. If CRC sectors are missing, decode the codewords which contain them first (C_j is in codeword $(S+j) \bmod L$). Data sectors whose Crc32 is not known yet are assumed to be correct if they are readable.

3. For every codeword: a data sector is an erasure if it is unreadable (in image files: contains the dead sector marker of dvdisaster, section 2.3) or its Crc32 does not match. Zero sectors

are known. With at most m erasures the codeword is decoded, otherwise its erased sectors stay unrecoverable. The data sector Crc32 values are checked after decoding.

4. After a complete repair, the MD5 sum of the data must equal $dataMD5$.

The decoder only needs one B_s-byte shard per layer and codeword at a time; an implementation processes W consecutive codewords per pass to read the image in larger pieces.

2.14 Supplementary sets (informative)

Error correction data can be added to an image later, without changing the data or the existing sets: a *supplementary set* is an ordinary RS04 set, normally an ecc file, which protects the same data as an existing set. It has its own $setID$ and may use other parameters (e.g. a different redundancy). It protects either exactly the same bytes (equal $dataBytes$ and $dataMD5$, e.g. a second ecc file for the image), or a superset: an ecc file created from an augmented image protects the data and the added zone of the augmented set.

The format does not link the sets; every set can be verified and decoded on its own. A decoder may combine all sets covering the same sectors. Since the codewords of the sets are different (other L, k and m), a sector recovered by one set is no longer an erasure for the others:

1. Decode every set as in section 2.13, writing the recovered sectors back to the image (and to the ecc files).

2. If some set recovered sectors and some codewords were not decodable, repeat step 1, as the other sets now see fewer erasures.

This way damage can be repaired which exceeds the capacity of each single set. A sector which remains unrecovered is reported once, however many sets could not recover it. The prognosis for the combined sets is obtained by applying the same iteration to the erasure maps: every codeword with at most m erasures is assumed to be recovered until no set makes progress.

dvdisaster combines the sets of an ecc file given with `-e`, of further ecc files given with additional `-e` options (or found in a directory given with `-e`), and the augmented sets of the image which overlap the data of the first ecc file. A further ecc file is only used if it has the same sector size, does not reach beyond the end of the data of the first ecc file and of the augmented sets, and its CRC sectors match the image (as for the confirmation of candidates in section 2.11).

2.15 Reading media (informative)

While reading a medium, dvdisaster checks the data sectors of every set with the sector size of the image being read (the augmented sets on the medium, or an ecc file given for it) against their Crc32 values, and the CRC and parity sectors of augmented sets against their tags. Only the beginning of each added zone, up to the last CRC sector, is read in advance; a parity sector is checked with the tag in front of it, which is read just before it. Adaptive reading, which skips tags it does not need and may start reading behind a tag, reads that tag first. A parity sector whose tag is unreadable is assumed to be correct. Since a skipped tag is missing in the image,

a parity sector which fails its check is stored as a missing sector, so that it is an erasure for the repair.

Audio CDs (and other CDs read as 2352 byte sectors) are checked against an ecc file with $B = 2352$. The Crc32 values only match if the rip and the ecc file were made with the same read offset (section 2.12): this is assumed once a sector which is not digital silence matches. If the first 8 such sectors do not match, the CRCs are not compared for the rest of the rip, which verifying and repairing then align. Sets with $B = 2324$ can not be checked while reading, since the 2352 byte sectors read from the CD do not tell which of them make up the protected data.

Adaptive reading stops as soon as every codeword of every set has at most m erasures (sectors which could not be read, or whose Crc32 does not match). Tags are not part of the code and are recreated by the repair, but the image must remain recognizable (section 2.11): one valid tag among the last 256 sectors of every augmented set must have been read. Sectors between sessions are not read, and the last 2 sectors of a session (TAO run-out) are not needed if they belong to no set.

2.16 Versioning and compatibility

- $version$ identifies the format of tags, CRC sectors and the code. This specification defines version 1. A reader must refuse to process a set whose tags carry a different version and should tell the user that a newer dvdisaster is required. Incompatible changes increase the version.

- $layoutVariant$ identifies the mapping of codewords to sectors. 0 is the layout of this specification (contiguous layers, no permutation). Readers must refuse sets with an unknown layout variant.

- $sectorBytes$ values other than 2048, 2324 and 2352 must be refused.

- Unknown bits in $flags$ are ignored; flags never change how data is decoded.

- Reserved bytes (the unused tail of the tag and of CRC sectors) are written as zero and ignored when read.

- $creatorVersion$ is informational (diagnostics and workarounds for bugs of a particular dvdisaster version).

2.17 Limits and edge cases

Data size	$1 \le dataBytes < 2^{64}$. An empty image cannot be protected. Augmented images need $dataBytes = S \cdot B$.
Codeword	$1 \le m \le k$ and $\mathrm{NextPow2}(m) + k \le 65536$, hence $k \le 65535$ and $n \le 65536$; the FFT size is at most 65536.
Layer size	$L \ge 1$ and $kL \ge D$. There is no upper limit; with $k = 1$ a layer holds all protected sectors.
Small sets	No special cases. $L < 32$ is allowed (dvdisaster only uses it when $D < 32$ or $E < 32$); then one 32-sector unit may hit a codeword twice. A minimal set: $S = 1$, $L = 1$, $k = 2$, $m = 1$, $T = 2$, $Q = 2$, $A = 4$.
$k = 1$	$m = 1$; each parity sector is a copy of the original sector.
$m = 1$	XOR parity over the k originals.
Zero sectors	Up to $kL-D$ sectors (possibly whole layers) are virtual zeros; they are never stored.
Tags	At least one ($Q \ge 2$). The last group of the added zone may have fewer than $T - 1$ payload sectors.
Implementation	*Informative:* dvdisaster handles sets with S, kL and A below 2^{31} sectors (4 TiB of 2048-byte sectors) and treats tags describing larger sets as invalid.

2.18 Design rationale

Damage simulation. The layout was chosen with the damage simulator `contrib/rs04-layout-sim.py` (Python 3 and numpy):

```
python3 contrib/rs04-layout-sim.py --trials 200 > results.md
```

(`--quick` gives a short smoke run). It maps damage models onto sectors using the CLV geometry of CD-R, DVD and BD (outer edge bands on single and dual layer discs, scattered loss of 16/32-sector ECC units, band with decaying fringe, radial scratches, contiguous runs of 4096 sectors and 1 cm^2 spots) and counts the erasures per codeword for RS03 and for RS04 layouts with $n \approx 255 \ldots 65535$, with and without permutations inside layers. The following table shows the loss (in percent of all sectors) at which the probability of a complete recovery drops below 99%, for a parity share of $m/n = 32/255$ (MDS limit 12.5%):

Damage model, medium	RS03	$n = 1023$	$n = 4095$	$n = 16383$	$n = 65535$
outer band, BD-25	12.55	12.51	12.55	12.55	12.59
band, DVD-9 (opposite track path)	12.55	12.51	12.56	12.57	12.62
scattered units, CD-R 80 min	5.46	8.50	10.69	–	–
scattered units, DVD-5	5.27	8.61	10.59	11.78	–
scattered units, BD-25	4.72	8.48	10.49	11.59	12.14
band with fringe, BD-25	8.16	10.29	11.42	12.10	12.40
radial scratches, BD-25	5.76	8.80	10.85	11.62	12.25
runs of 4096 sectors, BD-25	5.87	9.82	11.84	12.34	12.56
spots, BD-25	6.11	9.21	10.96	11.77	12.26

Findings: (1) long codewords more than double the tolerable scattered loss and stay at the limit for edge damage; (2) permuting 32-sector blocks inside the layers is within the noise for $n \geq 4095$, worse for small n and for contiguous runs, and costs about $C/32$ seeks per repair, so RS04 uses no permutation and only reserves the $layoutVariant$ field; (3) repair time on a hard disk grows with n for a given memory budget (BD-25 with 4 GiB: about 4 minutes for $n \approx 1023$, 19 minutes for $n \approx 16323$ and 1.1 hours for $n \approx 65283$; about 1–2 minutes on an SSD), therefore n is chosen per image and stored in the tags; (4) dvdisaster issue #43 is not explained by aliasing between the RS03 stride and the disc revolution.

Other decisions.

- *Tags in the added zone at a fixed interval* instead of a header at one place: any surviving part of the added zone describes the layout, also on dual layer discs whose outer edge is in the middle of the LBA range. Tag 0 is where RS03 has its ecc header, directly behind the data.
- *Payload Crc32 values in the tags*: they cannot be in the CRC sectors, which are protected by the parity they would describe. The tags have room for them anyway.
- *CRC sectors as originals* (like the RS03 CRC layer): lost checksums are recovered by the code. They have no header since the tags carry the layout.
- *Virtual zero sectors and shard padding*: nothing is stored that can be computed, which makes the ecc file identical to the added zone.
- *Relative positions and a random set ID*: sessions can be augmented without knowing where they will be written, and tags of other or older sets are never mixed in.
- *One field for the data length* ($dataBytes$) instead of a sector count and the bytes in the last sector: both follow from it.
- *No medium fingerprint*: the Crc32 values of the data sectors in the CRC sectors identify the data more reliably than the MD5 sum of one sector.

2.19 Test vectors

Code. The originals of each vector are filled with the output of the generator splitmix64 started with $state = seed$ (each step: $state \leftarrow state + \texttt{0x9E3779B97F4A7C15}$, $z \leftarrow state$, $z \leftarrow (z \oplus (z \gg 30)) \cdot \texttt{0xBF58476D1CE4E5B9}$, $z \leftarrow (z \oplus (z \gg 27)) \cdot \texttt{0x94D049BB133111EB}$, output $z \oplus (z \gg 31)$, all modulo 2^{64}), each 64-bit output stored little-endian, filling original shards $0 \ldots k-1$ consecutively. The table gives the MD5 sum of the parity shards $0 \ldots m-1$ concatenated (shards in Leopard buffer format, section 2.3.1).

seed	k	m	bytes	MD5 of parity	case
1	2	1	64	`9b1fe76ef28bf3823a5e3cf97adf0901`	XOR
2	5	3	64	`74130cbd984417137639d2a582b9b4a1`	small*
3	100	31	128	`8c5370f6adebf2d28b46125fdd201dc8`	no powers of 2
4	1000	1000	64	`82c42efe91c5110bb60f9235e7bd3991`	$m = k$
5	3584	512	2048	`047ba4c7183fabee56efa1053807edae`	typical
6	49152	16384	64	`055b961a368a2f12c3b5526636eeaf31`	$M + k = 65536$

* stock `leo_encode()` would use GF(2^8) for this size. "bytes" is the shard size. These values were produced by `src/rs04-ff16.c`, whose output was compared with Leopard-RS (built with GF(2^{16}) only) and the high-rate encoder of `reed-solomon-simd`. Vectors 1–3 are also reproduced by `contrib/rs04-reference.py` directly from equation (1).

Format. The data of a format vector is the splitmix64 stream started with $seed$, truncated to $dataBytes$ bytes. All tags use $setID$ = 01 02 03 04 05 06 07 08, $creatorVersion = 0$ and $flags = 0$. The table gives the MD5 sums of the data and of the complete added zone (= the ecc file).

	seed	B	$dataBytes$	S	c	L	k	m	T	Q	A
F1	7	2048	81820	40	1	7	6	3	8	22	26
F2	8	2352	23520	10	1	4	3	2	4	9	12
F3	9	2324	11620	5	1	6	1	1	2	7	14

	MD5 of the data	MD5 of the added zone
F1	`ac8364a7c7b4fcac9126f9d120a2b243`	`ce7142e2d9767ac6642ea567df200f90`
F2	`b5175b33aa5f95bec590eec94cb8392d`	`84d948d015532a395e1ccdea5e19b62f`
F3	`225073ed5626b7225ec38cc288d1a6d2`	`28a4b07832bda689aaa9b4e1278333ca`

F1 has a partial last data sector and a padding sector, F2 exercises the sector to shard mapping, F3 the case $k = m = 1$ with $T = 2$. The first 96 bytes of tag 0 of F1 are:

```
0000: 2a 64 76 64 69 73 61 73 74 65 72 2a 52 53 30 34  *dvdisaster*RS04
0010: 01 00 00 00 00 08 08 00 00 00 00 00 ce bb 08 3b  ................
0020: 01 02 03 04 05 06 07 08 9c 3f 01 00 00 00 00 00  .........?......
0030: 07 00 00 00 00 00 00 00 06 00 00 00 03 00 00 00  ................
0040: 28 00 00 00 00 00 00 00 ac 83 64 a7 c7 b4 fc ac  (.........d.....
0050: 91 26 f9 d1 20 a2 b2 43 f1 75 b2 b2 c8 97 c7 92  .&.. ..C.u......
```

3 The RS03 codec

This section describes the data format of the dvdisaster RS03 Reed-Solomon codec. RS03 can store parity data either in a separate file or append it to the .iso image on the same medium. In contrast to its predecessors RS03 is fully multi-threadable. RS03 is expected to become the default codec soon after its introduction in dvdisaster 0.80.

3.1 Physical layout

Optical media are recorded as a single long spiral[1] of sectors which are indexed beginning with 0. The first sector lies at the innermost position of the spiral and numbering continues onward to the outside of the spiral.

Reed-Solomon encoding works best when errors are evenly distributed over all ecc blocks. Therefore we must strive to spread our ecc blocks evenly over the media surface. To facilitate such distribution, dvdisaster logically divides the medium into 255 units which are called "layers" for historical reasons. Figure 1 illustrates how a medium is divided into n data layers, one CRC layer, and m ecc layers, with $n + m + 1 = 255$. Ecc blocks are comprised by taking one

[1] Multiple layered media contain one spiral for each physical layer, but are otherwise conceptually identical.

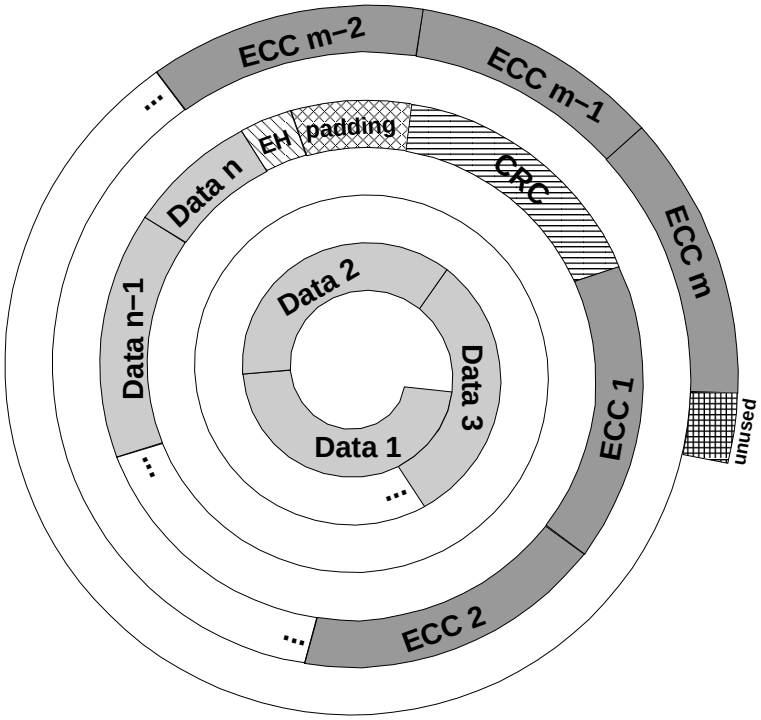


Figure 1: Physical RS03 layout

byte from each layer as shown in fig. 2 on the following page. This distributes the ecc block reasonably good over the medium surface.

Layer size is measured in numbers of sectors which are 2048 bytes in size. All 255 layers have the same size. The data layers map exactly to the iso image which is to be protected by dvdisaster; e.g. the number and sequence of sectors in $Data_1, \ldots, Data_n$ is the same as in the iso image. Two extra sectors are appended to the ISO image holding the ecc header "EH"; these are logically treated as a part of the ISO image. If the ISO image size plus the two EH headers is not an integer multiple of the layer size, the last (n-th) data layer will be padded accordingly.

The data layers are followed by a CRC layer. Each CRC layer sector contains a data structure holding CRC32 checksums for the data sectors plus additional parameters which were used during the RS03 encoding process. The data and CRC layers are protected by the Reed-Solomon parity which is stored in the remaining layers ($ECC_1, \ldots, ECC_m$). Since the medium capacity is not necessarily an integer multiple of 255, some unused sectors remain at the end of the medium. These are neither written nor referenced in any way.

The RS03 data can be stored either directly on the medium or into a separate file. Figure 1 shows the first case where ecc data is embedded into the image; this is also called a "RS03 augmented ISO image" in dvdisaster terminology. In the other case a separate error correction file will be created containing the sectors starting with the EH header. The following discussion is based on the augmented image case; see section 3.7 for handling the file based format.

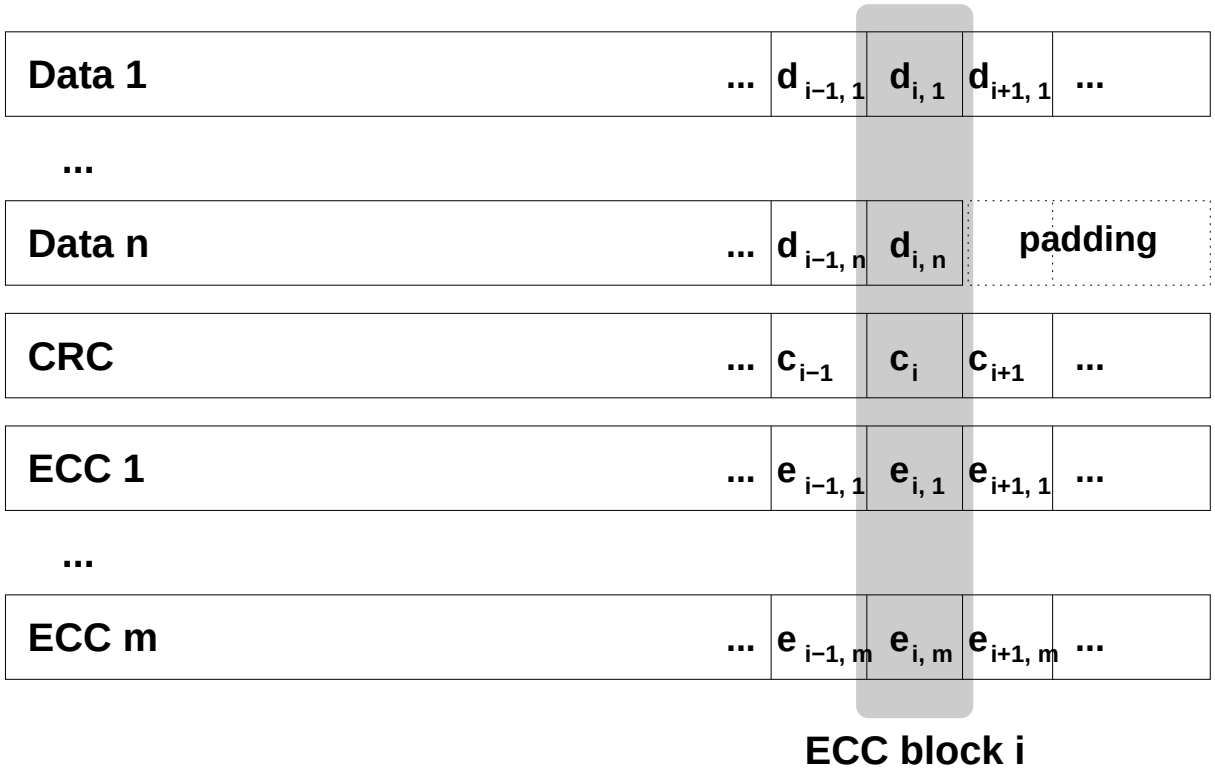


Figure 2: Logical RS03 layout

3.2 Logical layout

The relationship between layers and ecc blocks is stated again in the logical view presented in figure 2. Here the 255 layers are shown in stacked order. From each layer the $i-th$ byte corresponds to the $i-th$ error correction block. The parity is calculated using n data bytes $d_{i,1}, \ldots, d_{i,n}$ plus the crc byte c_i. The resulting m roots of the completed Reed-Solomon code are then stored in the ecc bytes $e_{i,1}, \ldots, e_{i,m}$.
Since the input iso image plus the two EH sectors is usually not an integer multiple of the layer size, the last data layer $data_n$ may contain padding sectors containing a special signature. The content of the padding sectors is used in the ecc byte calculation and is written into the augmented image.

3.3 Calculating the layout for encoding

When encoding with RS03 the layout of the augmented image is fully specified by two values: the maximum medium size and the iso image size (from now on, "size" always means "number of 2048K sectors").

Media sizes are hard coded and taken from table 1. Since we need to divide the medium into 255 layers, the layer size is:

$$layer\ size = \left\lfloor \frac{medium\ size}{255} \right\rfloor$$

Medium type	Maximum size	Layer size
CD	359.424	1.409
DVD 1 layer	2.295.104	9.000
DVD 2 layers	4.171.712	16.359
BD 1 layer	11.826.176	46.377
BD 2 layers	23.652.352	92.754

Table 1: Sector size parameters for several media types

This allows us to compute the number of data layers needed to cover the iso image plus the ecc header:

$$data\ layers = \left\lceil \frac{iso\ image\ size + 2}{layer\ size} \right\rceil$$

The number of padding sectors in the last data layer is:

$$padding\ sectors = layer\ size * data\ layers - iso\ image\ size - 2$$

For Reed-Solomon encoding, we will have to encode

$$n\ data\ bytes = data\ layers + 1$$

and produce the following number of parity bytes:

$$m\ roots = 255 - n\ data\ bytes$$

The RS03 augmented image must fill the medium completely (except for the $medium\ size \bmod 255$ sectors at the end). However for performance reasons the maximum redundancy is capped to 200%, or 170 roots. This means that the ISO image must at least span the first 255-170=85 layers, otherwise additional padding will be added to fill up the 85 data layers. This situation is not reflected in the calculations and figure shown above.

3.4 Re-calculating the layout from defective media

In order to recover a defective medium, the values of *layer size* and *data layers* need to be determined. The RS03 format allows for three heuristics with increasing complexity for learning about these values:

3.4.1 Using the Ecc Header

All required information can be obtained from the data structures of the Ecc Header which is described in appendix A. If ecc data is stored in a separate error correction file, the first 4096 bytes of the ecc file yield the Ecc Header. Otherwise, let n be the size of the ISO file system which can be obtained from the ISO file system master block. Then the Ecc Header is typically found in the RS03-augmented image at sectors $n, n+1$ or at $n+150, n+151$ (due to padding inserted by some popular CD-R mastering software).

If the ISO file system master block is unreadable, the Ecc Header can be identified by its characteristic signature and checksum. If the Ecc Header is encountered during reading of the defective medium it might be worthwhile to generate a tentative ISO master block in the image file. This would speed up future processing of the image; however current implementations of dvdisaster do not yet implement this feature.

3.4.2 Using the CRC layer

Each CRC layer sector contains a data structure which not only holds the CRC32 checksums but also a copy of important parameters from the Ecc header (see section 3.5 for details). CRC sectors can be easily recognized by looking for their signature and checksum while scanning the medium image. If dvdisaster finds a valid CRC sector and the Ecc header is defective, a tentative Ecc header is written to the image to speed up further operations on the image file.
However it should be noted that since all CRC sectors are stored consecutively on the medium, they can easily be wiped out by a large defective region on the medium. Therefore, another heuristic exists for learning about the RS03 layout.

3.4.3 Evaluating the Reed-Solomon code

If neither the Ecc Header nor any CRC sectors are readable the RS03 layout can be determined by the following heuristic.
First, the medium size is determined from table 1. This is always possible as long as the drive will recognize the medium at all. Since the layer size is $\left\lfloor \frac{medium\ size}{255} \right\rfloor$, the location of the 255 layers on the medium is now known. The remaining task is to find out the redundancy of the Reed-Solomon code, e.g. how many layers contain roots for the RS code.

Taking the *i-th* sector from each layer will produce a valid error correction block, but with unknown redundancy. As RS03 will create redundancies using 8 to 170 roots, we employ a brute-force approach by evaluating the Reed-Solomon code for $8..170$ roots. If the error correction is successful for n roots and the sector from layer *255-1-n* yields the CRC data structure, the correct number of roots has been found.

In reality, not all 162 combinations of roots need to tested since additional information can be exploited:

1. If the sector from layer *255-1-n* is present/readable, we do not need to test for n roots any further: Encoding with n roots would have produced a CRC sector in this place.

2. If the number of erasures (as indicated by unreadable sectors) is higher than n, we can trivially skip the RS decoding. We might have to test another set of 255 sectors though if testing for all other numbers of roots fails as well.

Criterion 1) should quickly narrow down the possible numbers of roots in the average case, e.g. when enough redundancy is available for recovering the medium. Worst case behaviour of trying each ecc block for 8..170 roots is likely to appear only when the medium is unrecoverable, e.g. when more sectors are damaged than the Reed-Solomon code can correct.

3.5 Contents of the CRC layer

Each sector of the CRC layer contains the data structure shown in appendix B. Following the numbering from figure 2, CRC sector c_i contains the CRC32 checksums for data sectors $d_{j,1}, \ldots, d_{j,n}$ with $j = (i+1) \bmod layer\ size$. The purpose of this offset is to have the error correction of ECC block i recover the CRC checksum for the next ECC block $i+1$. In case of readable but corrupted sectors this will keep the error correction in erasure mode and therefore save precious redundancy (the RS code can recover twice as much errors when the location of defective data is known).

Checksums for data sector $d_{j,k}$ are stored in array element CrcBlock->crc[k]. Unused array elements are set to zero. The remaining contents of the CRC sector structure provide configuration and layout information; see appendix B for details.

3.6 Encoding the ecc layers

Encoding the error correction information requires reading and buffering of at least 255 sectors comprising the ecc block (see fig. 2 for a definition of the ecc block). A possible encoding algorithm might process each ecc block at a time. For each ecc block i it would do the following:

First, the n data sectors $d_{i,1}, \ldots, d_{i,n}$ of the ecc block are read in. The CRC layer sector c_i is initialized, filled in with checksums generated by processing the previous ecc block, and completed by calculating its own checksum *selfCRC*. Unused portions of c_i remain zero. Afterwards the CRC32 checksums of $d_{i,1}, \ldots, d_{i,n}$ are calculated and stored away using the same buffering mechanism. Since the hand-over of CRC checksums between ecc blocks is the only place where RS03 does not fully parallelize, data sector I/O and CRC32 caching needs to be carefully thought out in multithreaded implementations.

Once $d_{i,1}, \ldots, d_{i,n}$ and c_i have been prepared, 2048 sets of a RS(255,k) code (with $k = 255 - n - 1$) are calculated by looping over the 2048 bytes of the ecc sectors. If l denotes a certain byte position between $0, \ldots, 2047$ in the ecc block sectors, then the *l-th* byte from $d_{i,1}, \ldots, d_{i,n}, c_i$ is retrieved and fed into the RS(255,k) encoder. The resulting parity bytes $p_1, \ldots, p_m$ are stored in byte position l of the ecc layer sectors $e_{i,1}, \ldots, e_{i,m}$. When all 2048 bytes of the ecc block sectors have been processed the ecc layer sectors can be written out; either into the error correction file or into the RS03 augmented image.

The RS(255,k) encoder is the same for RS01, RS02 and RS03. See appendix D for the parameters used in the encoder.

3.7 Encoding as a separate error correction file

If the image size is too close to the medium capacity, not enough space is left for augmenting the image with redundancy. dvdisaster will refuse to augment images when there is insufficient space for at least 8 roots. Creating images with less than 43 roots (20% of redundancy) will trigger a warning that the error correction capacity may be too low. In those cases, storing the error correction information in a separate file comes as an alternative.

RS03 error correction files ("ecc files") contain the same error correction information and layout as in the augmented image case, with the following differences:

Omittance of data padding sectors. While the image format shown in figures 1 and 2 may contain padding sectors between the ecc header and the CRC layer, those sectors are not written into the ecc file. The padding sectors are however required during encoding and decoding, e.g. they need to be virtually created in memory when processing the respective ecc blocks. Therefore an ecc file providing *nroots* of redundancy will contain *2 + (nroots+1) * layer size* sectors. Physically it will contain the ecc header, then the CRC layer and finally the *nroots* ecc layers. In contrast to the augmented image case, the ecc header *is not part of an ecc block* and can therefore not be recovered by the error correction. If the ecc header is lost in an ecc file, its contents can be reconstructed from a still existing block in the CRC layer and then be rewritten accordingly.

Freely chooseable redundancy. In the augmented image case the redundancy is always chosen to fill up the medium completely. For ecc files the redundancy can be freely chosen by the user between 8 roots (3.2%) and 170 roots (200%). Encoding with more than 170 roots is technically possible, but run-time requirements get out of proportion; hence the selectable redundancy is capped at 200%.

As a consequence of the variable redundancy the ecc file layout can only be determined by looking at the ecc header or CRC sectors. The strategy of experimentally evaluating the Reed-Solomon code (see sub section 3.4.3) however can not be applied to ecc files since neither the size of the padding area nor the original size of the possibly truncated image and ecc files can be determined.

To see whether this is really a limiting factor we look at the typical outcome of recovering a single file from a defective medium:

- The ecc file is fully read, but random sectors are damaged.
- The ecc file is truncated to the position of the first read error.

In both scenarios it is highly likely that at least one CRC sector survives at the beginning of the file; in that case the error correction will not only recover the image but also repair the ecc file into its original state.

Although this gives RS03 ecc files good chances to remain functional even when being partly damaged, it is highly recommended to store ecc files only on media which are themself being protected by dvdisaster. ISO and UDF file systems do not have sufficient redundancy for their meta data (e.g. directory structures). If such meta data becomes unreadable a significant number of files may become completely inaccessible. Please note that this is a general weakness of file-based data protection and recovery: The meta data is not part of any file and can therefore not be protected by any error correction data put inside the file(s). This is the also the simple reason why we did not use tools like PAR2 and developed dvdisaster instead; the image-based approach of dvdisaster protects both files and meta data.

4 The RS02 codec

This section describes the dvdisaster RS02 Reed-Solomon codec. It was developed during the winter of 2005/2006 in order to facilitate augmenting iso images directly with error correction data.
RS02 is based on the Reed-Solomon encoders and decoders introduced with RS01, but focuses exclusively on augmenting iso images. The allocation of data sectors within an ecc block follows a similar scheme as in RS01. However the layout of the parity bytes is vastly different between RS01 and RS02, as the codec must cope with any parity sector being damaged or unreadable. Consequently a RS02 image can lose as many sectors as allowed by the redundancy of the error correction data, and the lost sectors can be any combination of data and parity sectors, as it is expected from a Reed-Solomon scheme.

Unlike RS01, which will be completely superseded by RS03 soon, the case of RS02 vs. RS03 still remains open, as both codecs have their individual strengths. RS02 is slightly more space efficient than RS03, so on CD media RS02 might provide slightly more redundancy (typically one additional root) than RS03. This effect will be less pronounced on larger media like DVD and BD. RS02 images can be augmented to an arbitrary size which may be smaller than the maximum medium size, while RS03 requires augmenting the image to the full medium size. This might favour RS02 for working on images which are only 30% or less of the medium size, as they can be encoded with less than the maximum of 170 roots (the maximum redundancy requires lots of time to compute, producing a three-fold redundancy which may not be needed in all cases). On the other hand RS03 will counter the performance argument since it can encode at least 20 times faster than RS02 on multi-core architectures, because RS02 encoding can not be parallelized. See the end of section 5 for a speed comparison of RS01 vs. RS03; RS01 and RS02 are very similar performance-wise. Finally, the data layout of RS03 does not depend on interspersed ecc headers which gives it a better robustness over RS02; see subsection 4 for details.

4.1 Physical layout

RS02 must be applied to the .iso image before it is written to the medium. Additional sectors are appended to the .iso image containing the parity data. The data structures of the .iso image are not changed to reflect the new image size, so the original part of the augmented .iso image remains untouched. The parity sectors can be removed from the augmented image by simply truncating the .iso image to its original sector size; the resulting image file will have the same contents as prior to the augmentation. As a side effect, the parity data is invisible to applications reading the medium at the filesystem level, including most hardware media players. If you find a player which gets confused by media containing RS02 (or RS03) parity, please consider telling the dvdisaster project about it. As of this writing, not a single device has been reported to run into problems with the RS02 data scheme. The RS02 augmented image might conflict with optical media writing software, though. If the writing software decides the image length by looking at the iso filesystem structures, the parity data portion of the image might not be written to the medium. Most current writing programs do however measure the .iso image by examining its file size, and will transfer the parity data correctly. To be sure you should follow the steps described under “Testing image compatibility” at the dvdisaster site

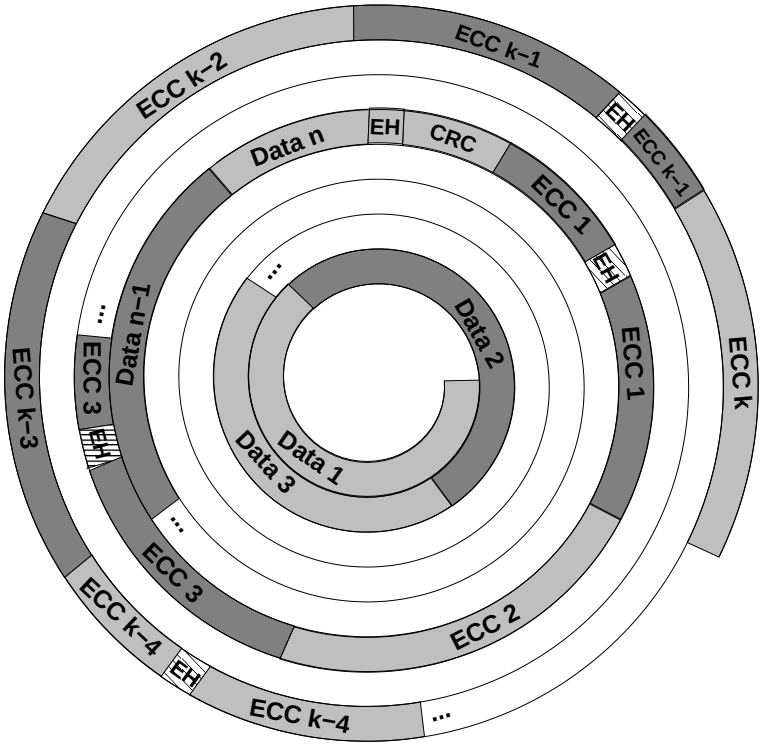


Figure 3: Physical RS02 layout

(https://web.archive.org/web/20180428070843/http://dvdisaster.net/en/how
once before using each version of your optical media authoring software.
Like the other dvdisaster codecs, RS02 is based on a RS(255,k) Reed-Solomon code with each ecc block being comprised of n data bytes and k parity bytes, and $n + k = 255$. The n data bytes comprise the .iso image which will be written to the medium, and the additional ecc header and CRC checksums added by dvdisaster. Reed-Solomon encoding works best when errors are distributed evenly over all ecc blocks. Therefore we must strive to distribute the ecc blocks evenly over the medium surface. To facilitate such mapping, dvdisaster logically divides the medium into 255 logical units which are called "layers" for historical reasons. Figure 3 shows how the medium is divided into n data layers and k ecc layers, with $n + k = 255$. Ecc blocks are created by taking on byte from each layer as shown in fig. 4 on the following page. This distributes the ecc block reasonably good over the medium surface. All layers have the same length in bytes, with the possible exception of data layer n. As the .iso image size plus the size of one ecc header and the CRC data is usually not a multiple of the layer size, the n-th data layer may be shorter than the layer size and considered to be filled up with a virtual zero padding. The zero padding is not written out to the augmented image (note that data layer n is intentionally drawn shorter in fig. 3), but it is used in the calculation of the respective parity bytes.

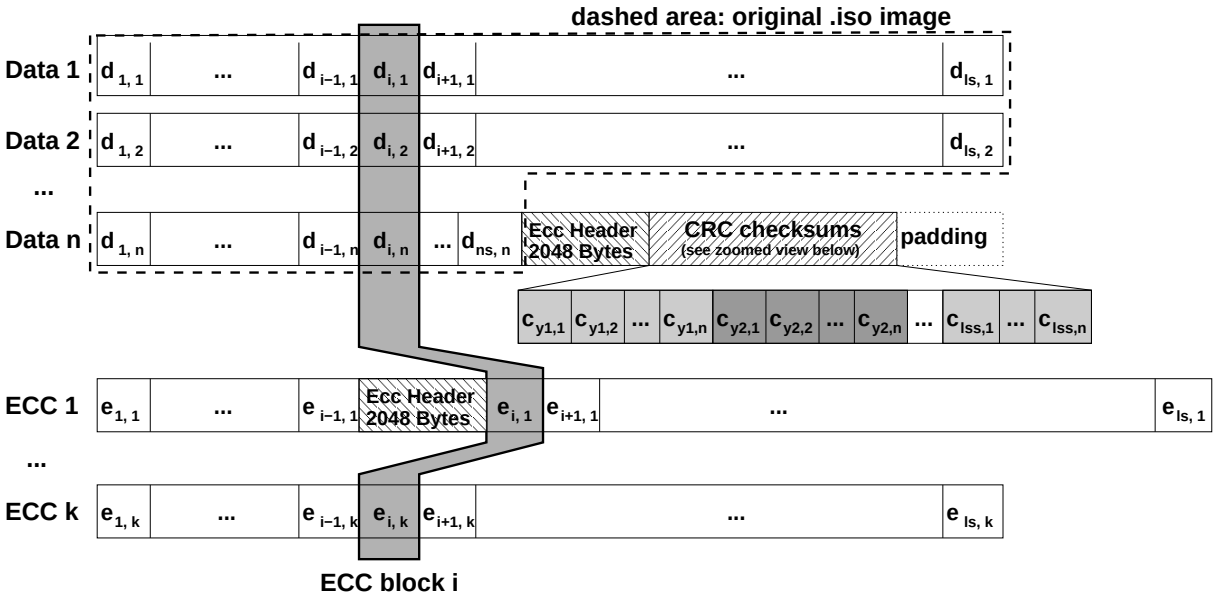


Figure 4: Logical RS02 layout

4.2 Logical layout

The data layout in the augmented image is shown in figure 4. Note that in this figure the data is byte-indexed; e.g. $d_{1,1}$ denotes the first data byte in the augmented image. Each layer has a length of ls bytes, with the exception of data layer n which may be shortened (see subsection 4.3.1 for an exact calculation of its size). Some ecc layers my be interleaved with redundant copies of the ecc header. The ecc header size is not included in the respective ecc layer size.

Data layers. A data layer index $d_{i,j}$ refers to the i-th byte in the j-th data layer. The n data layers are mapped in a linear fashion to the original iso image. $d_{1,1}$ maps to the first .iso image byte, and $d_{ns,n}$ maps to the last .iso image byte (ns is the number of remaining iso image bytes in the n-th data layer).

The last data layer is special because it does not only contain the rest of the iso image, but also the ecc header and the CRC checksums. These extensions are logically treated as a part of the iso image; their contents are used in the ecc data calculation and are therefore protected by the ecc data. The ecc header follows immediately after byte $d_{ns,n}$ and is 4096 bytes long. Its format is described in appendix A. For RS02, only the data fields marked with "all" or "RS02" are relevant; all other fields should be set to zero.

Data layer n does also contain the CRC32 checksums of each data sector up to the ecc header. If the .iso image contains s sectors, then the CRC field contains $4s$ bytes, rounded up to the nearest multiple of 2048. CRC32 checksums are calculated over a whole CD sector comprising 2048 bytes. Let $c_{y,j}$ be the 4-byte checksum of the y-th sector in the j-th layer and lss be the number of sectors in each layer. Then $c_{y,j}$ = CRC32($d_{2048*y,j}$, $d_{2048*y+1,j}$, ..., $d_{2048*y+2047,j}$).

y_1 is usually not the first sector in the layer, but a later sector. In general, $y_i = (i + offset) \bmod lss$. The offset is introduced to restore the CRC32 sums of ecc block $i+1$ during the correction of ecc block i. This helps if the data portion of the image is corrupted with wrong byte values

and the sectors containing the CRC32 sums have been lost. The error correction will start at the ecc block i which is determined by the offset, and whose CRC32 checksums are stored in the ecc header (at least one ecc header will be recovered before any error correction can begin). Correcting ecc block i will recover the CRC32 checksums for ecc block $i+1$ in the image (and possibly some more in advance, as less than 2048 bytes are required for one set of checksums). This makes it possible to detect corrupted bytes by the checksums and flag them as erasures which effectively doubles the error correction capabilities of the Reed-Solomon code.

Ecc layers. For an image augmented with k roots, the parity bytes will be spread over k ecc layers. In order to calculate the first ecc block, bytes $d_{1,1}$ to $d_{1,n}$ are taken from the n data layers. The RS(255,k) code is calculated (see appendix D for its parameters) and the resulting k parity bytes $e_{1,1}$ to $e_{1,k}$ are stored in the k ecc layers. The next ecc blocks are calculated and stored accordingly; ecc block i is marked grey in the figure. Care must be taken to honour the non-linear mapping of ecc layer bytes as the ecc area is interleaved with 20-40 copies of the ecc header. The ecc header copies are placed at sector addresses whose numbers are large powers of two. This makes it possible to heuristically search for them during the decoding boot-strap process when no other information (image size, layer size, etc.) is yet available. See section 4.5 on the search heuristics and section 4.6 on calculating ecc bytes positions from the non-linear mapping.

4.3 Calculating the image layout for encoding

The image layout can be either computed automatically to fill up the medium as much as possible, or by user selected criteria such as a maximum image size or a specified redundancy.

4.3.1 Automatic layout calculation

The only available inputs to automatic layout detection are the .iso image size and a table of maximum media sizes (see tab. 1 in the RS03 section for the respective values). From the media size table the smallest possible medium is chosen which can contain the .iso image. In some border cases, with e.g. the .iso image being only 100 sectors smaller than the medium capacity, the automatic layout calculation will fail later due to insufficient space on the medium. In such cases, the user must decide between choosing the next larger medium size or splitting the image contents onto two media by himself (splitting a 700MiB CD image onto two CDs may be better than writing it to a DVD).

From now on, all calculations are given in numbers of 2048K sectors or sectors addresses unless noted otherwise. The number of sectors required for the CRC checksums can be directly computed from the .iso image size:

$$crc\ sectors = \left\lceil \frac{4 * iso\ image\ sectors}{2048} \right\rceil$$

The total accumulated size of all data layers is the sum of the .iso image size, the number of crc sectors and the two sectors required for the ecc header. Since these sectors are protected by the parity, they are called *protected sectors*:

$$protected\ sectors = iso\ image\ size + 2 + crc\ sectors$$

These calculations also produce two important sector addresses within the augmented image:

- The sector with address *iso image size* marks the location of the ecc header; and
- The sector at address $iso\ image\ size + 2$ marks the beginning of the CRC checksum data.

The next step is to partition the *protected sectors* and the remaining medium space into an optimal layer size. It is carried out iteratively.
For an approximate start, we determine the free space on the medium:

$$free\ space = medium\ capacity - protected\ sectors$$

and estimate a preliminary value for the number of roots and data layers:

$$k\ roots = min\left(170, \quad \left\lfloor \frac{255 * free\ space}{medium\ capacity} \right\rfloor\right)$$

$$n\ data = 255 - k\ roots$$

The maximum number of roots is capped at 170 which is approximately a three-fold redundancy. Larger values would get too computationally expensive.
The preliminary layer size is then:

$$preliminary\ layer\ size = \left\lceil \frac{protected\ sectors}{n\ data} \right\rceil$$

and the expected size of the parity layers is:

$$preliminary\ ecc\ size = k\ roots * preliminary\ layer\ size$$

From these values we compute a 2^p which has about 20-40 multiples in the *preliminary ecc size* address space. This value will be used for interleaving the ecc header copies with the ecc layers:

$$p = \left\lceil log_2\ \frac{preliminary\ ecc\ size}{40} \right\rceil$$

Now the chosen values might be actually too big since we haven't taken the ecc header copies into account. So the final task is to add up the number pf parity sectors and ecc header copies. If these fit into the free medium space, we are done; otherwise the calculations are done again with one root less.

while($n\ roots > 7$)

$layer\ size := \left\lceil \frac{protected\ sectors}{n\ data} \right\rceil$

$ecc\ size := n\ roots * layer\ size$

$first\ ecc\ header\ repeat\ addr := \left\lceil \frac{protected\ sectors}{2^p} \right\rceil * 2^p$

$space\ for\ interleaved\ sectors := protected\ sectors + ecc\ size - first\ ecc\ header\ repeat\ addr$

$number\ of\ ecc\ copies := \left\lfloor \frac{space\ for\ interleaved\ sectors}{2^p - 2} \right\rfloor + 1$

$total\ added\ sectors := 2 + crc\ sectors + ecc\ size + 2 * number\ of\ ecc\ copies$

If $iso\ image\ sectors + total\ added\ sectors < medium\ size$, we have a valid layout: STOP.

Otherwise, set $n\ roots := n\ roots - 1$ and $n\ data := 255 - n\ roots$ and do another iteration.

The iteration will either terminate with a valid layout or fail when $n\ roots$ drops below the minimum redundancy of 8 roots.

4.3.2 Layout calculation by user selected criteria

The user has several means of specifying a certain redundancy:

Specifying the maximum number of sectors for the augmented image. This case is simply handled by setting *medium capacity* to the user selected sector size rather than using the maximum medium size from the built-in table. Afterwards, calculations continue as described in section 4.3.1.

Specifying the number of roots to use. In this case we can skip the calculations for *free space* and *k roots* as described in section 4.3.1, and instead set *k roots* directly to the user selected value. Then the layout calculation proceeds as usual.

Specifying the percentage of redundancy to use. For a given number of *k roots*, the resulting redundancy in percent is:

$$\frac{k\ roots \cdot 100}{255 - k\ roots}$$

Pick a suitable value for *k roots* so that the user selected value is met or slightly exceeded. Proceed with the given number of roots as described in the previous paragraph.

4.3.3 Layout calculation from ecc header information

In a given ecc header struct *eh*, the number of sectors in the .iso image is recorded as *eh->sectors* and the number of roots is contained in *eh->eccBytes*. Calculation of the layout is done as shown in section 4.3.1, with the exception of omitting the calculation for *free space* and setting *k roots* directly to *eh->eccBytes*.

4.4 Automatic layout calculation example

Let's assume we are going to encode an .iso image of 295.000 sectors. This is well below the CD medium capacity of 359.424 sectors, so we start with:

$medium\ capacity = 359.424\ sectors$
$iso\ image\ size = 295.000\ sectors$

The number of CRC sectors will be:

$crc\ sectors = \left\lfloor \frac{4*295.000}{2.048} \right\rfloor = 577\ sectors$

The total size of all data layers is:

$protected\ sectors = 295.000 + 2 + 577 = 295.579\ sectors$

The next step is creating some preliminary starting values:

$free\ space = 359.424 - 295.579 = 63.845\ sectors$
$k\ roots = min\left(170, \left\lfloor \frac{255*63.845}{359.424} \right\rfloor\right) = min(170, 45) = 45\ roots\ (or\ layers)$
$n\ data = 255 - 45 = 210\ layers$

Now some more preliminary values can be computed:

$preliminary\ layer\ size = \left\lceil \frac{295.579}{210} \right\rceil = 1.408\ sectors$
$preliminary\ ecc\ size = 45 * 1.408 = 63.360\ sectors$

Finally, we compute $p = 11$ since $\frac{63360}{2^{11}} = 30,9$.

Now the chosen values must be verified to produce a layout which is still smaller than the image size. We compute (the first two values are already known):

$layer\ size = \left\lceil \frac{295.579}{210} \right\rceil = 1.408\ sectors$
$ecc\ size = 45 * 1.408 = 63.360\ sectors$
$first\ ecc\ header\ repeat\ addr = \left\lceil \frac{295.579}{2048} \right\rceil * 2048 = 296.960$
$space\ for\ interleaved\ sectors = 295.579 + 63.360 - 296.960 = 61.979\ sectors$
$number\ of\ ecc\ copies = \left\lfloor \frac{61.979}{2048-2} \right\rfloor + 1 = 31\ header\ repeats$
$total\ added\ sectors = 2 + 577 + 63.360 + 2 * 31 = 64.001\ sectors$

This layout will generate an augmented image containing $295.000 + 64.001 = 359.001$ sectors which is less than the medium capacity of 359.424 sectors and therefore accepted.

4.5 Re-calculating the layout from defective media

In order to recover a defective medium, at least one ecc header must remain readable and be located by the following heuristic. This is a major difference to RS03, which has more and different means for bootstrapping the recovery (see section 3.4 for details). Once one ecc header has been recovered, the ecc data layout can be calculated as described in section 4.3.3. From this point, the error correction is done using the parameters and data described in section 4.8.

If the medium is not damaged or only slightly damaged, the following short cut might work: The size of the .iso image can be determined from the iso file system header. Then the ecc header immediately following the .iso image part of the augmented image is either located at

sector number $iso\ image\ size$ or $iso\ image\ size + 150$. The latter case arises because some popular CD authoring software appends 150 padding sectors to any .iso image it creates.

If the short cut does not work due to the required sectors being damaged, the following strategy is employed. The size of the augmented image can always be determined; it can either be queried from the drive or it is the file size of a file-based image. Then apply the following algorithm:

$p = \lfloor log_2(image\ size) \rfloor$
while $p > 32$
  $pos = \left\lfloor \frac{image\ size}{2^p} \right\rfloor \cdot 2^p$
  while $pos > 0$
    if *sector at pos* is a valid ecc header: STOP.
    if *sector at pos* is unreadable, set $pos := pos - 2^p$.
      Continue with inner while loop.
    if *sector at pos* is readable and not a ecc header, set $p := p - 1$.
      Continue with outer while loop.

In order to test for a valid ecc header, check that *ec->cookie* equals the 16-byte string "*dvdisaster*RS02". Then check that the CRC32 sum of the ecc header matches the value recorded in *eh->selfCRC*, with *eh->selfCRC* set to the byte sequence 0x47,0x50,0x4c,0x00 for the purpose of calculating the CRC32 sum.

Please notice that during testing of the sectors at multiples of $2^{(p-1)}$, all sectors previously tested for 2^p will be examined again. It is therefore highly recommended to cache results from previous iterations of the outer while loop, especially when reading sectors from the optical medium.

4.6 Sector addressing and initialization scheme

For encoding and decoding purposes it is required to retrieve the *i-th* sector from the *j-th* data or ecc layers, e.g. to calculate the corresponding sector number in the augmented image. The reverse calculation is also needed, e.g. to calculate the corresponding layer and sector index for a given sector number in the augmented image.

Bear in mind that as shown in figure 4, an augmented image is divided into two logical parts. There is a data area containing the .iso image contents, the first ecc header and the CRC checksums. The data area is protected by the parity in the ecc area, which contains the parity data interleaved with copies of the ecc header.

In order to carry out the calculations described below, the following values from the layout calculation (see section 4.3.1 are required:

protected sectors	the size of the data part in sectors
layer size	the number of sectors per layer
2^p	the modulo value for locating ecc header copies

Converting (layer, sector index) pairs into image sector numbers. The *i-th* sector of data layer *j* has the following address s in the image:

$$s = j \cdot layer\ size + i$$

If $s >= protected\ sectors$, s is a padding sector which must not be read from the image file, but created in memory (see the paragraph on initialization below).

To calculate the sector address es of the *i-th* sector from the *j-th* ecc layer, the non-linear mapping of the ecc sectors has to be taken into account. The index of the first interleaved ecc header is:

$$first\ interleaved = \left\lceil \frac{protected\ sectors}{2^p} \right\rceil \cdot 2^p$$

Since *protected sectors* is equal to the address of the first ecc sector in the image, the amount of ecc sectors before the first interleaved ecc header is:

$$base\ ecc\ sectors = first\ interleaved - protected\ sectors$$

The ecc sector we are looking for would have the following index if ecc sectors were linearly mapped:

$$ecc\ index = j \cdot layer\ size + i$$

If *ecc index* $<$ *base ecc sectors*, es = *protected sectors* + *ecc index*. Otherwise, the non-linear mapping must be taken into account. The number of interleaved ecc headers before the (currently unknown) sector position es is:

$$interleaved\ headers = \left\lfloor \frac{ecc\ index - base\ ecc\ sectors}{2^p - 2} \right\rfloor$$

Therfore the position of the ecc sector in the augmented image is:

$$es = protected\ sectors + ecc\ index + 2 \cdot interleaved\ headers + 2$$

Example. To continue the example from section 4.4, the position of the 17th ecc sector in the 3rd ecc layer shall be computed. The relevant layout values are:

protected sectors	=	295.579
layer size	=	1.408
2^p	=	2.048

The first interleaved ecc header is at position:

$$first\ interleaved = \left\lceil \frac{295.579}{2.048} \right\rceil \cdot 2.048 = 296.960$$

Before the first interleaved ecc header,

$$base\ ecc\ sectors = 296.960 - 295.579 = 1.381$$

ecc sectors have been stored. The linear index of the sought ecc sector is:

$$ecc\ index = 3 \cdot 1.408 + 17 = 4.241$$

Since $4.241 \geq 1.381$, the embedded ecc headers must be taken into account. There are

$$interleaved\ headers = \left\lfloor \frac{4.241 - 1.381}{2.048 - 2} \right\rfloor = 1$$

interleaved ecc headers, each containing 2 physical sectors. Therefore the position of the sought ecc sector in the image is:

$$es = 295.579 + 4.241 + 2 + 2 = 299.824$$

Converting image sector numbers into (layer, sector index pairs).
If the sector number $s <$ *protected sectors*, the sector will map to the data part as follows:

$$layer = \lfloor s\ /\ layer\ size \rfloor$$

$$i = s \bmod layer\ size$$

Otherwise, the mapping to the ecc part is calculated as follows. The index of the first interleaved ecc header is:

$$first\ interleaved = \left\lceil \frac{protected\ sectors}{2^p} \right\rceil \cdot 2^p$$

If $s\ mod\ 2^p \leq 1$, the sector maps to the *n-th* interleaved ecc header, with:

$$n = \left\lfloor \frac{s - first\ interleaved}{2^p} \right\rfloor$$

If $s < first\ interleaved$, the sector is an ecc parity sector with the following mapping:

$$layer = \lfloor (s - protected\ sectors)\ /\ layer\ size \rfloor$$

$$i = (s - protected\ sectors)\ mod\ layer\ size$$

If $s \geq first\ interleaved$, the mapping of the ecc parity sector is calculated as follows:
The amount of ecc sectors before the first interleaved ecc header is:

$$base\ ecc\ sectors = first\ interleaved - protected\ sectors$$

The number of interleaved ecc headers before sector s is:

$$interleaved\ headers = \left\lfloor \frac{s - first\ interleaved - 2}{2^p} \right\rfloor$$

If ecc sectors were mapped linearly, then s had the linear index:

$$ecc\ index = s - protected\ sectors - 2 \cdot interleaved\ headers - 2$$

Finally, this means that s maps to the following parity sector:

$$layer = \lfloor ecc\ index\ /\ layer\ size \rfloor$$

$$i = ecc\ index \bmod layer\ size$$

Padding sectors. Let *iso image size* be the size of the .iso image prior to augmenting it with error correction data. In order to augment the image with error correction sectors, the following sectors are treated as padding sectors which are filled with zeroes:

- All sectors $s >$ *protected sectors*.
- The first ecc header (sectors $iso\ image\ size$ and $iso\ image\ size + 1$).

The first ecc header sectors must be treated as padding to break a circular dependency with the parity bytes; as the ecc header contains a md5 sum over all parity bytes, it can not be used as input for the parity generation.

4.7 Encoding the checksums

For each sector of the .iso image a CRC32 checksum is calculated and stored in the data part of the augmented image (see fig. 4). By using the conventions of section 4.2, let $d_{i,j}$ be the i-th byte of the j-th data layer and $c(y, j)$ the 4-byte checksum of the y-th sector in the j-th data layer. Then $c(y, j)$ = CRC32($d_{2048*y,j}$, $d_{2048*y+1,j}$, ..., $d_{2048*y+2047,j}$).

Let $first\ layer\ crc\ idx = (iso\ image\ size + 2) \bmod layer\ size$.

n is the number of data layers.

A total of $\left\lceil \frac{iso\ image\ size}{512} \right\rceil$ sectors holding the CRC32 checksums must be generated. The checksums are sorted by the layer sector y first, then by layer number i. So for each layer sector y, there is a block of n checksums generated, and there are $layer\ size$ blocks of checksums total. Checksum generation does not start with layer sector 0, but rather with layer sector $first\ layer\ crc\ idx$. Subsequent blocks are generated in ascending layer sector order *modulo layer size* so that all *layer size* layer sector positions are eventually covered. This scheme produces the following sequence of checksums:

$$
\begin{array}{l}
c((first\ layer\ crc\ idx + 1) \bmod layer\ size, \quad 1) \\
c((first\ layer\ crc\ idx + 1) \bmod layer\ size, \quad 2) \\
\ldots \\
c((first\ layer\ crc\ idx + 1) \bmod layer\ size, \quad n) \\
\hline
c((first\ layer\ crc\ idx + 2) \bmod layer\ size, \quad 1) \\
c((first\ layer\ crc\ idx + 2) \bmod layer\ size, \quad 2) \\
\ldots \\
c((first\ layer\ crc\ idx + 2) \bmod layer\ size, \quad n) \\
\hline
\ldots \\
\hline
c((first\ layer\ crc\ idx + layer\ size - 1) \bmod layer\ size, \quad 1) \\
c((first\ layer\ crc\ idx + layer\ size - 1) \bmod layer\ size, \quad 2) \\
\ldots \\
c(first\ layer\ crc\ idx \bmod layer\ size, \quad n - 1^*) \\
\hline
c(first\ layer\ crc\ idx \bmod layer\ size, \quad 1) \\
c(first\ layer\ crc\ idx \bmod layer\ size, \quad 2) \\
\ldots \\
c(first\ layer\ crc\ idx \bmod layer\ size, \quad n - 1^*)
\end{array}
$$

*) The last sectors of each data layer may be padding sectors. For those padding sectors, *no* CRC32 checksums are generated and stored (e.g. the number of generated checksums is always exactly *iso image size*).
Since *iso image size* is usually not a multiple of 512, the last sector in the data part may only be partially filled with checksum data. The remaining bytes of this sector must be filled with the repeated byte sequence 0x47,0x50,0x4c,0x00 which is the ASCII string representation of the text "GPL".

A copy of the CRC32 sums for the layer sectors at position ($first\ layer\ crc\ idx \bmod layer\ size$) is stored in the ecc header, starting there at byte position 2048. This has the advantage that the CRC checksums are already available for the *first layer crc*-th sectors of data layers $1, \ldots, n$. Any corrupted bytes in those sectors are detected by the CRC32 and can be handled by the error correction in erasure mode, saving precious parity bytes. When the error correction has restored all sectors of the *first layer crc*-th ecc block, note that the *first layer crc*-th sector of data layer n will contain the CRC32 checksums for the data sectors in the next ecc block (*first layer crc + 1*). Therefore the layout is robust against loss of CRC sectors as they are restored by the error correction just before they are actually needed.

4.8 Encoding the ecc layers

Encoding the ecc layers requires the following steps:

First the image must be examined whether it does already contain augmented ecc data (either RS02 or RS03). If ecc data is found, the image must be stripped to the original size of the .iso image. Nesting ecc data is not supported by the current codecs and it might derail the heuristics for detecting the augmented data properly. From a technical point, nesting ecc data does not make sense either.

Next the image must be checked for missing sectors, and be rejected if it is incomplete. Producing and writing images with missing sectors to a medium is confusing to the user as dvdisaster will always report the medium as partially readable even though it does not contain any physical defects. Also the error correction will never succeed for such media as it is just restoring the sector in its missing state. During the check for missing sectors the CRC32 checksums of each sector can be computed as described in section 4.7 and, after writing a placeholder for the first ecc header, be appended to the image. Also, the MD5 sum of the .iso image can be calculated at this time and kept for insertion into the ecc header field *ec->mediumSum*. As another step of preparation, enough space should be appended to the image to store the ecc layer sectors. This makes sure that the encoder does not run out of disk space during its potentially lenghty work, and minimizes the impact of fragmentation due to random writes into the appended image area under most file systems.

Finally, the error correction information needs to be encoded. Please refer to fig. 4 on the location of the bytes comprising an error correction block. Although the ecc blocks could be encoded by a byte-wise scheme, a possible encoding algorithm would preferably buffer at least the 255 sectors holding the required data for 2048 subsequent ecc blocks, and process those in bulk. From the first n data layers, the required bytes are retrieved and fed into the RS(255,k) Reed-Solomon encoder, with $k = 255 - n$. The RS(255,k) encoder is the same for RS01, RS02 and RS03. See appendix D for the parameters used in the encoder.

Please refer to the previous section on information about zeroed-out and zero-padded data sectors. The resulting k parity bytes are distributed into the k ecc layers. When writing out the ecc data into the image, free gaps must be left for the interleaved ecc headers; see section 4.6 for information on calculating the interleaved ecc header positions. At this time, the MD5 sums of each ecc layer can be updated incrementally.

When all parity sectors have been calculated, the ecc headers can be completed by filling in their *eh->eccSum* field. This field contains the MD5 sum calculated over the MD5 sums over each of the k ecc layers. In contrast to a single MD5 sum spanning the ecc layers in a linear fashion, this approach allows for an incremental calculation of the MD5 sum while the ecc data is generated and written out.

5 The RS01 codec

This section describes the dvdisaster RS01 Reed-Solomon codec. It was conceived during the summer of 2004 for creating error correction files in the first dvdisaster versions. At this time, CD media was still predominant. Typical machines were based on Pentium 4 (tm) processors. Measured by todays standards physical RAM and hard disk space were scarce, and especially hard disk random I/O was extremely slow.

In order to work efficiently with the available technology, RS01 was designed to be as space efficient as possible and to minimize hard disk random access. Optimizing the data layout for random access efficiency lead to a parity byte distribution which left the error correction file vulnerable to being damaged. RS01 was occasionally being critcized for not being able to recover from damaged error corrction files, but these points were not really fair. RS01 error correction files were never designed for being stored on fragile media. They are supposed to be either stored on hard disk, or to be stored on optical media which itself is protected by dvdisaster error correction which has the following consequences: Unlike optical media, hard disks do not degrade gradually. Hard disks are usually either 100% readable or completely dead, so we can assume that error correction files on hard disk are either completely readable or fully lost.

Storing error correction files on optical media is a different story. While an error correction file could protect itself to some degree against lost sectors (as RS03 ecc files do), it is still prone to the shortcomings of a file level error correction. The biggest disadvantage of file level error correction is that there is no protection of file system meta data. If meta data like a directory node becomes damaged, all files in the directory are lost regardless of the redundancy contained within the files. Therefore any medium containing error correction files must be protected with an image level error correction layer (by using RS01,RS02 or RS03 on the medium), since only image level error correction avoids meta data sectors to become a single point of failure. See the discussion at `https://web.archive.org/web/20180428070843/http://dvdisaster.n` for more information on the advantages of image level data protection over file level approaches.

Nevertheless, the time has come to phase out the RS01 codec. Consider creating an error correction file with 32 roots for a 650MiB sized image using both codecs[2]:

codec	ecc file size	encoding time
RS01	94.58MiB	46.2s
RS03	96.68MiB	2.4s

RS03 is about 2.2% less storage efficient than RS01 since its data layout has been rearranged for better parallelization. But this is made up by a 19-fold speed improvement as RS03 can use multiple cores and SSE2 extensions (of course the speed improvement varies depending on the hardware used). Since all other properties of RS03 do at least match those of RS01, it's fair to begin phasing out RS01 in dvdisaster.

dvdisaster V0.80 will be the first and only version featuring all three codecs. In version 0.82, users will be presented a note the RS01 became deprecated. In subsequent releases support for encoding RS01 will be removed. Of course, capabilities to use and decode RS01 will remain in dvdisaster for umlimited time. Existing RS01 error correction files should remain in use and there is be no need to replace them with RS03 ones.

[2]The benchmark was done using the GNU/Linux version of dvdisaster 0.79.4 on a AMD Athlon(tm) II X4 615e processor. RS03 used all 4 cores of the machine. Both image and ecc files were stored in `/dev/shm` to rule out I/O effects.

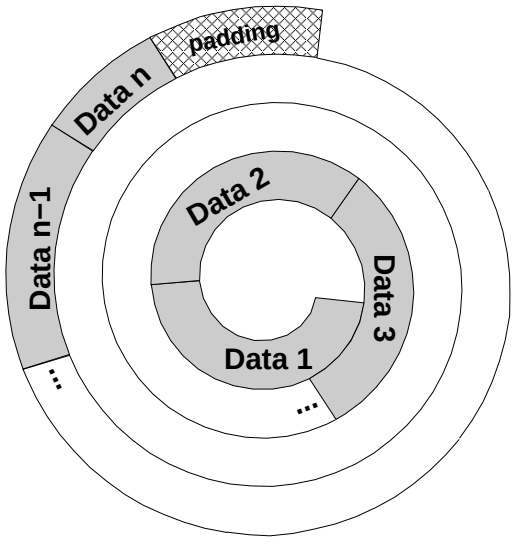


Figure 5: Interpretation of physical layout in the .iso image

5.1 Physical layout

RS01 is meant to protect data which has already been written to an optical medium, so the parity data can not be appended to the medium and must instead be kept in a separate error correction file. Like all dvdisaster codecs, RS01 is based on a RS(255,k) Reed-Solomon code with each ecc block being comprised of n data bytes and k parity bytes, and $n + k = 255$.
The n data bytes are taken from an iso image generated from the medium. Reading data directly from the optical drive during encoding would slow down the process tremendously due to massive random access over the medium, and quickly wear out the drive mechanics. However producing the .iso image takes one fast linear read, accesses the drive in a way it is designed to be used, and puts the data on hard disk which can sustain the needed random access I/O.
Reed-Solomon codes work best when errors are evenly distributed over all ecc blocks. Therefore the n data bytes used for creating an ecc block must be picked from locations which are evenly distributed over the medium with a maximum distance between each data byte pair. To obtain a suitable data distribution, it is taken into account that optical media are recorded as a single long spiral[3] of sectors each containing 2048 bytes. The first sector lies at the innermost position of the spiral and is indexed with 0; numbering continues onward to the outside of the spiral. The .iso image contains a 1:1 mapping of this storage scheme, with the first 2048 bytes holding the contents of sector 0, the next 2048 bytes resembling sector 1, and so on.
When encoding with n data bytes per ecc block, the iso image is divided into n layers which physically map to the medium as shown in fig.5. This distributes the ecc block reasonably good over the medium surface. However since the image size does not need to be a multiple of the layer size, the n-th layer may be physically shorter as the layer size. For encoding purposes, the non-existant sectors in layer n are treated as sectors being filled with 2048 zero bytes.

[3]Multiple layered media contain one spiral for each physical layer, but are otherwise conceptually identical.

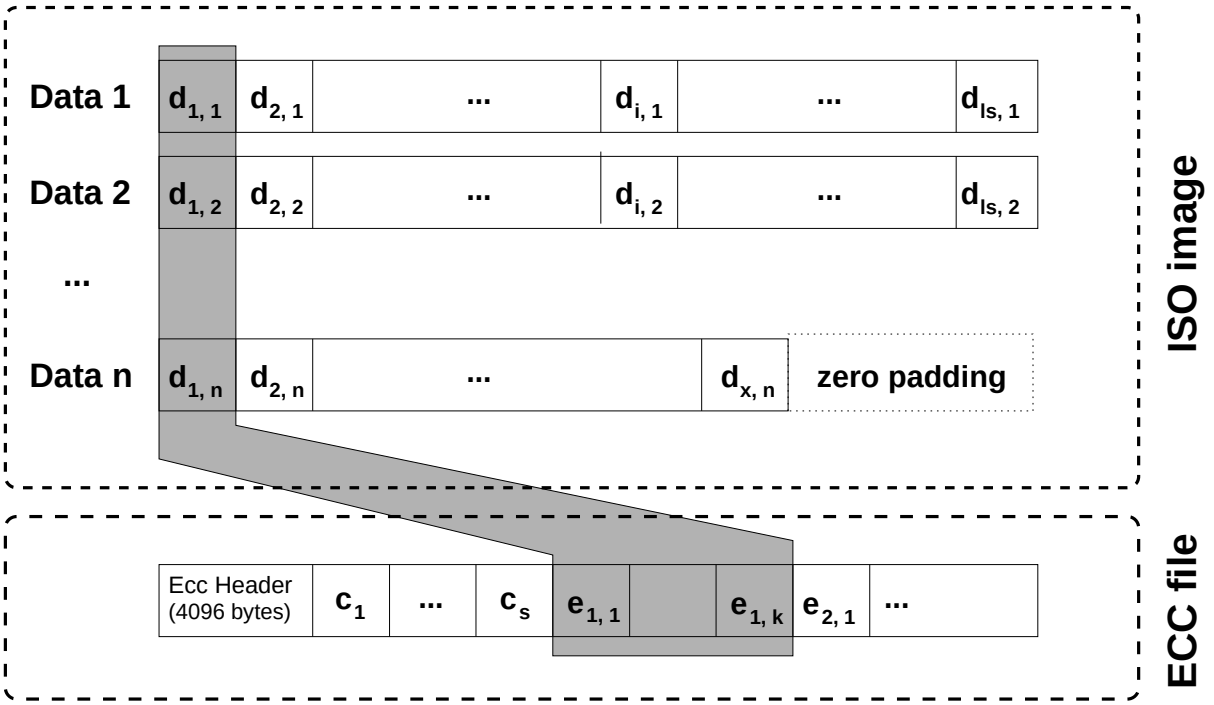


Figure 6: Logical RS01 layout

5.2 Logical ecc file layout

The ecc file layout, and therefore the relationship between the iso image contents and the ecc file, is shown in figure 6. The first 4096 bytes of the ecc file contain the ecc header whose format is described in appendix A. For RS01, only the data fields marked with "all" or "RS01" are relevant; all other fields should be set to zero.

Next to the ecc header comes the CRC section of the ecc file. If the iso image contains s sectors, the next $4 * s$ bytes in the ecc file contain the CRC32 sums of the sectors from the iso image: Let $b_1, \ldots, b_{2048}$ denote the bytes of the first data sector; $b_{2049}, \ldots, b_{4096}$ those of the second data sector and so on. Then $c_1 = CRC32(b_1, \ldots, b_{2048})$, $c_2 = CRC32(b_{2049}, \ldots, b_{4096})$ etc. Note that in contrast to RS02 and RS03, bytes from the CRC section are not included into the ecc block calculation and are therefore not protected by ecc.

The remainder of the ecc file contains the parity bytes of the ecc blocks. For an ecc file built with k roots, the iso image is logically divided into $n = 255 - k$ layers as shown in figure 6. The $d_{i,j}$ denote the $i - th$ byte in the $j - th$ layer. In order to create the first ecc block, bytes $d_{1,1}$ to $d_{1,n}$ are taken from the n layers. Then the RS(255,k) code is calculated (see appendix D for its parameters) and the resulting k parity bytes $e_{1,1}$ up to $e_{k,1}$ are stored in the ecc file. The resulting ecc block is marked grey in the figure. The next ecc blocks are calculated and stored accordingly. In total, the ecc section contains $k * ls$ bytes of parity information, with the k parity bytes of each ecc block being stored consecutively.

5.3 Calculating the layout for encoding

The RS01 layout is fully determined by the number of roots for the error correction code and the iso image size in sectors (from now on, "size" always means "number of 2048K sectors). The number of roots can be freely chosen by the user from the range of $[8...100]$. The iso image size is directly measured from the iso image file.

The number of data layers is simply calculated from the number of roots, k:

$$data\ layers = 255 - k$$

The size of each layer is:

$$layer\ size = \left\lceil \frac{medium\ size}{data\ layers} \right\rceil$$

At the end of the last layer, $data\ layers * layer\ size - medium\ size$ zero filled padding sectors are used in the encoding process.

5.4 Getting the layout when recovering defective media

The required parameters are taken from the ecc header stored in the error correction file (see appendix A). Especially, the number of roots are taken from the *eccBytes* field and the medium size is recorded in the *sectors* field.

5.5 md5 checksums

RS01 provides two md5 checksums for integrity checking. The md5 sum of the iso image is calculated and stored in the *mediumSum* field of the ecc header. Another md5 sum is calculated over the ecc file, excluding the first 4096 bytes, and stored in the *eccSum* field of the ecc header. It can be used to verify the integrity of the ecc file itself. The ecc header is protected by its own CRC checksum which is stored in the *selfCRC* field.

The md5 checksum generation is the major obstacle for parallelizing the encoder. In RS03, md5sum generation has been made optional since the RS03 layout allows suffcient consistency checks by doing a quick error syndrome check using the Reed-Solomon code.

5.6 Special cases

Error correction files can be created for any type of input files, not just iso files, as long as the input files are "reasonably" long[4]. Since input files are processed in units of 2048 kByte sectors, files whose byte size is not an integer multiple of 2048 are virtually padded with zeroes. In that case, the *inLast* field of the ecc header contains the real byte size of the last file "sector" so that recovering the last file sector does not write out the padding bytes. A size of zero in the *inLast* field means that the last sector contains 2048 bytes.

[4]Input files should contain at least 2048*(255-k) bytes, so that there is at least one sector for each data layer.

A The common Ecc header format

The ecc header is defined in the include file *dvdisaster.h*. Its C definition is as follows:

```
typedef struct _EccHeader
{   gint8 cookie[12];          /* "*dvdisaster*" */
    gint8 method[4];           /* e.g. "RS01" */
    gint8 methodFlags[4];      /* 0-2 for free use by the respective methods; 3 see above */
    guint8 mediumFP[16];       /* fingerprint of FINGERPRINT SECTOR */
    guint8 mediumSum[16];      /* complete md5sum of whole medium */
    guint8 eccSum[16];         /* md5sum of ecc code section of .ecc file */
    guint8 sectors[8];         /* number of sectors medium is supposed to have w/o ecc*/
    gint32 dataBytes;          /* data bytes per ecc block */
    gint32 eccBytes;           /* ecc bytes per ecc block */
    gint32 creatorVersion;     /* which dvdisaster version created this */
    gint32 neededVersion;      /* oldest version which can decode this file */
    gint32 fpSector;           /* sector used to calculate mediumFP */
    guint32 selfCRC;           /* CRC32 of EccHeader (currently RS02 only) – since V0.66 –*/
    guint8 crcSum[16];         /* md5sum of crc code section of RS02 .iso file */
    gint32 inLast;             /* bytes contained in last sector */
    guint64 sectorsPerLayer;   /* layer size for RS03 */
    guint64 sectorsAddedByEcc;/* sectors added by RS02 */
    gint8 padding[3960];       /* pad to 4096 bytes: room for future expansion */
} EccHeader;
```

The ecc header is used in all ecc formats (RS01, RS02, RS03) of dvdisaster, but not all fields apply to all formats. See the following table for the meaning and usage of the fields:

Field	Usage	Format(s)
cookie	Magic byte sequence for recognizing the header. Contains the string `*dvdisaster*`.	all
method	4 characters describing the format; currently allowed: `RS01`, `RS02`, `RS03`.	all
methodFlags	4 bytes for further specification of the format.	
	Byte 0 contains the following flag: Bit 0 - The *mediumSum* field is valid. Bit 1 - Set to 1 in ecc files.	 RS03 RS03
	Bytes 1-2 are unused in the current methods.	
	Byte 3 contains the following flags: Bit 0 - ecc data was created by a development release. Bit 1 - ecc data was created by a release candidate. If these bits are present, the user will be hinted that he is using ecc data from a non-stable dvdisaster version.	all

(continued on next page)

$mediumFP$	The md5sum of the sector specified by the *fpSector*. The sector should be chosen to have a huge probability being unique to the medium; currently sector 16 (the ISO filesystem root sector) is used.	all
$mediumSum$	The md5sum of the ISO image. For RS01 this is the md5sum of the whole image; for RS02 it is calculated for the original ISO image (without the added RS02 sectors). RS03 uses this value only when bit 1 in *methodFlags* is set.	all
$eccSum$	On RS01 this is the md5sum of the ecc file excluding the first 4096 bytes. For RS02 this is the md5sum calculated over the md5sums of the $nroots$ ecc layers. RS03 does not use this value.	RS01, RS02
$sectors$	For error correction files this is the number of sectors in the protected medium. If augmented images are used, this denotes the number of sectors in the original ISO image (without the added RS02/RS03 sectors).	all
$dataBytes$	The number of data layers, including the CRC layer.	all
$eccBytes$	The number of ecc layers (= number of roots) for the parity. $dataBytes + eccBytes = 255$.	all
$creatorVersion$	The dvdisaster version used for creating this ecc data. A decimal value 102345 would mean dvdisaster version 10.23.45.	all
$neededVersion$	The minimum dvdisaster version required for processing this ecc data. Version encoding as above.	all
$fpSector$	The sector used for calculating $mediumFP$.	all
$selfCRC$	A CRC32 checksum of the ecc header itself. Not used header fields are set to zero and the selfCRC field is initialized to the value 0x4c5047 (little endian).	
$crcSum$	md5sum of the CRC layer in RS02 encoded images.	RS02
$inLast$	The number of Bytes contained in the last image sector. This allows for encoding of files with arbitrary length, not just ISO images. dvdisaster versions prior to V0.66 do not use this field and always assume it to be 2048 which is the default for iso images.	all
$sectorsPerLayer$	The number of sectors per layer.	RS03
$sectorsAddedByEcc$	The total number of sectors (Headers, CRC, ECC) added.	RS02
$padding$	The ecc header is zero padded to a length of 4096 bytes. Future codes may allocate additional space for the zero padding. See the note below for usage of the upper 2048 bytes on RS02/RS03.	all
Byte 2048-4096	A copy of the first CRC layer sector.	RS02

B RS03 CRC block format

The crc layer contains 2048 byte blocks containing the data structure described below. Except for the CRC32 checksums most of the information contained in this data structure is copied from the Ecc Header described in appendix A. The crc block format is defined in the include file *dvdisaster.h* and has the following C definition:

```
typedef struct _CrcBlock
{   guint32 crc[256];          /* Checksum for the data sectors */
    gint8 cookie[12];          /* "*dvdisaster*" */
    gint8 method[4];           /* e.g. "RS03" */
    gint8 methodFlags[4];      /* 0-2 for free use by the respective methods; 3 see above */
    gint32 creatorVersion;     /* which dvdisaster version created this */
    gint32 neededVersion;      /* oldest version which can decode this file */
    gint32 fpSector;           /* sector used to calculate mediumFP */
    guint8 mediumFP[16];       /* fingerprint of FINGERPRINT SECTOR */
    guint8 mediumSum[16];      /* complete md5sum of whole medium */
    guint64 dataSectors;       /* number of sectors of the payload (e.g. iso file sys) */
    gint32 inLast;             /* bytes contained in last sector */
    gint32 dataBytes;          /* data bytes per ecc block */
    gint32 eccBytes;           /* ecc bytes per ecc block */
    guint64 sectorsPerLayer;   /* for recalculation of layout */
    guint32 selfCRC;           /* CRC32 of ourself, zero padded to 2048 bytes */
} CrcBlock;
```

The CrcBlock data structure is used in the CRC layer of RS03 augmented images only. RS02 has a similar CRC layer but uses a different concept for retrieving layout information from the image. The following table describes the meaning and usage of the CrcBlock fields:

Field	Usage
crc	If this data structure is found in the *i-th* sector of the CRC layer, it contains the CRC32 checksum for data sectors $d_{j,1}, \ldots, d_{j,n}$, with $j = (i+1) \bmod \ layer\ size$. See figure 2 for details. Please note that the crc[] array is filled starting from crc[0], and unused field are left zero.
$cookie$	Magic byte sequence for recognizing the header. Contains the string `*dvdisaster*`.
$method$	4 characters describing the format; currently only "RS03" may appear here.

(continued on next page)

$methodFlags$	4 bytes for further specification of the format. Byte 0 contains the following flags: Bit 0 - The *mediumSum* field is valid. Bit 1 - Set to 1 in ecc files. Bytes 1-2 are unused in the current methods. Byte 3 contains the following flags: Bit 0 - ecc data was created by a development release. Bit 1 - ecc data was created by a release candidate. If these bits are present, the user will be hinted that he is using ecc data from a non-stable dvdisaster version.
$creatorVersion$	The dvdisaster version used for creating this ecc data. A decimal value 102345 would mean dvdisaster version 10.23.45.
$neededVersion$	The minimum dvdisaster version required for processing this ecc data. Version encoding as above.
$fpSector$	The sector used for calculating $mediumFP$.
$mediumFP$	The md5sum of the sector specified by the *fpSector*. The sector should be chosen to have a huge probability being unique to the medium; currently sector 16 (the ISO filesystem root sector) is used.
$mediumSum$	The md5sum of the original ISO image if the first bit in the *methodFlags* field is set. Since md5sum generation can not be parallelized, the user may opt not to calculate this checksum if multi core encoding is used.
$dataSectors$	For error correction files this is the number of sectors in the protected medium. If augmented images are used, this denotes the number of sectors in the original ISO image (without the added padding and RS03 sectors).
$inLast$	The number of Bytes contained in the last image sector. This allows for encoding of files with arbitrary length, not just ISO images.
$dataBytes$	The number of data layers, including the CRC layer.
$eccBytes$	The number of ecc layers (= number of roots) for the parity. $dataBytes + eccBytes = 255$.
$sectorsPerLayer$	The number of sectors per layer.
$selfCRC$	A CRC32 checksum of the ecc header itself. Not used fields are set to zero and the selfCRC field is initialized to the value 0x4c5047 (little endian).
remaining bytes	The CrcBlock is zero padded to a size of 2048 bytes.

C RS04 tag and CRC sector format

A tag sector holds the following 2048-byte structure; for $B > 2048$ the remaining bytes of the sector are zero. All integers are little-endian (section 2.1). Every field is at an offset which is a multiple of its size, so the C structure below has no padding on any common ABI.

Offset	Size	Type	Field	Contents
0	12	char	$cookie$	`*dvdisaster*` (no terminating zero)
12	4	char	$method$	`RS04`
16	2	u16	$version$	1 (section 2.16)
18	1	u8	$layoutVariant$	0
19	1	u8	$flags$	bit 0: created by a development release; bit 1: created by a release candidate. The user is hinted that the data comes from a non-stable dvdisaster version. Other bits are zero.
20	2	u16	$sectorBytes$	B: 2048, 2324 or 2352
22	2	u16	$tagInterval$	T: 2, 4, . . . , 256
24	4	u32	$creatorVersion$	dvdisaster version which created the set; the decimal value 102345 means version 10.23.45
28	4	u32	$selfCRC$	Crc32 of bytes 0–2047 computed with this field set to zero
32	8	bytes	$setID$	random; identical in all tags of the set
40	8	u64	$dataBytes$	length of the protected data in bytes; $S = \lceil dataBytes/B \rceil$
48	8	u64	$layerSectors$	L
56	4	u32	$dataLayers$	k
60	4	u32	$eccLayers$	m
64	8	u64	$ownPosition$	relative position of this tag, $S + gT$ for tag g
72	16	bytes	$dataMD5$	MD5 of the $dataBytes$ bytes of data
88	1020	u32[255]	crc	Crc32 of the following payload sectors (section 2.7); entries $T-1 \ldots 254$ and entries beyond the last payload sector are zero
1108	940			zero

A CRC sector holds 512 little-endian Crc32 values (2048 bytes, section 2.6), followed by $B - 2048$ zero bytes.

C definitions (from `src/rs04-includes.h`; integers must be converted on big-endian hosts):

```
/* RS04 on-disc structures, see documentation/codec-specs/rs04.tex.
   rs04-common.c reads and writes them byte-wise; all integers are
   little-endian on disc, independent of the host byte order. */

#include <glib.h>
#include <stddef.h>

#define RS04_VERSION           1    /* tag format version */
#define RS04_LAYOUT_VARIANT    0    /* contiguous layers, no permutation */
#define RS04_CRCS_PER_SECTOR 512    /* Crc32 values in one CRC sector */
#define RS04_MAX_TAG_INTERVAL 256   /* T: one tag every T added-zone sectors */

#define RS04_FLAG_DEVEL  (1<<0)     /* created by a development release */
#define RS04_FLAG_RC     (1<<1)     /* created by a release candidate */
```

```
typedef struct _RS04Tag
{  gint8   cookie[12];      /*    0: "*dvdisaster*" */
   gint8   method[4];       /*   12: "RS04" */
   guint16 version;         /*   16: RS04_VERSION; refuse others */
   guint8  layoutVariant;   /*   18: RS04_LAYOUT_VARIANT; refuse others */
   guint8  flags;           /*   19: RS04_FLAG_*; unknown bits are ignored */
   guint16 sectorBytes;     /*   20: B = 2048, 2324 or 2352 */
   guint16 tagInterval;     /*   22: T, power of two, 2..256 */
   guint32 creatorVersion;  /*   24: e.g. 7910 = dvdisaster 0.79.10 */
   guint32 selfCRC;         /*   28: Crc32 of bytes 0..2047, this field = 0 */
   guint8  setID[8];        /*   32: random, identical in all tags of a set */
   guint64 dataBytes;       /*   40: protected bytes; S = ceil(dataBytes/B) */
   guint64 layerSectors;    /*   48: L */
   guint32 dataLayers;      /*   56: k */
   guint32 eccLayers;       /*   60: m */
   guint64 ownPosition;     /*   64: S + a, relative to the set start */
   guint8  dataMD5[16];     /*   72: md5 of the dataBytes data bytes */
   guint32 crc[RS04_MAX_TAG_INTERVAL-1]; /* 88: Crc32 of the following
                                            tagInterval-1 payload sectors */
   guint8  zero[940];       /* 1108: always zero */
} RS04Tag;                  /* 2048 bytes; rest of a B > 2048 sector is zero */

/* CRC sector j: Crc32 of data sectors 512*j .. 512*j+511, unused entries 0 */

typedef struct _RS04CrcSector
{  guint32 crc[RS04_CRCS_PER_SECTOR];
} RS04CrcSector;

G_STATIC_ASSERT(sizeof(RS04Tag) == 2048);
G_STATIC_ASSERT(offsetof(RS04Tag, dataBytes) == 40);
G_STATIC_ASSERT(offsetof(RS04Tag, crc) == 88);
G_STATIC_ASSERT(sizeof(RS04CrcSector) == 2048);
```

D RS(255,k) encoding parameters and examples

dvdisaster uses a standard, non-shortened Reed-Solomon code with the following commonly used parameters:

The Galois field tables are generated by the field generator polynomial 0x187 ($1 + X + X^2 + X^7 + X^8$). The Reed-Solomon code generator polynomial is created using element 0x70 as first consecutive root and the primitive element 0x0b.

As a starting point for testing your own implementation, some values and tables are shown below. The logarithm and anti-logarithm tables in the Galois field are shown in tables 2 and 3. Please note that there is no need for hard-coding these tables as their contents can be enumerated by using the field generator polynomial.

When encoding for 32 roots, the RS code generator polynomial will be:
01 5b 7f 56 10 1e 0d eb 61 a5 08 2a 36 56 ab 20 71 20 ab 56 36 2a 08 a5 61 eb 0d 1e 10 56 7f 5b 01
or in index form:
00 f9 3b 42 04 2b 7e fb 61 1e 03 d5 32 42 aa 05 18 05 aa 42 32 d5 03 1e 61 fb 7e 2b 04 42 3b f9 00

Using the above generator polynomial for encoding the data byte sequence $\{0, 1, \ldots, 222\}$ produces the following parity bytes:
2f bd 4f b4 74 84 94 b9 ac d5 54 62 72 12 ee b3 eb ed 41 19 1d e1 d3 63 20 ea 49 29 0b 25 ab cf

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	ff	00	01	63	02	c6	64	6a	03	cd	c7	bc	65	7e	6b	2a
10	04	8d	ce	4e	c8	d4	bd	e1	66	dd	7f	31	6c	20	2b	f3
20	05	57	8e	e8	cf	ac	4f	83	c9	d9	d5	41	be	94	e2	b4
30	67	27	de	f0	80	b1	32	35	6d	45	21	12	2c	0d	f4	38
40	06	9b	58	1a	8f	79	e9	70	d0	c2	ad	a8	50	75	84	48
50	ca	fc	da	8a	d6	54	42	24	bf	98	95	f9	e3	5e	b5	15
60	68	61	28	ba	df	4c	f1	2f	81	e6	b2	3f	33	ee	36	10
70	6e	18	46	a6	22	88	13	f7	2d	b8	0e	3d	f5	a4	39	3b
80	07	9e	9c	9d	59	9f	1b	08	90	09	7a	1c	ea	a0	71	5a
90	d1	1d	c3	7b	ae	0a	a9	91	51	5b	76	72	85	a1	49	eb
a0	cb	7c	fd	c4	db	1e	8b	d2	d7	92	55	aa	43	0b	25	af
b0	c0	73	99	77	96	5c	fa	52	e4	ec	5f	4a	b6	a2	16	86
c0	69	c5	62	fe	29	7d	bb	cc	e0	d3	4d	8c	f2	1f	30	dc
d0	82	ab	e7	56	b3	93	40	d8	34	b0	ef	26	37	0c	11	44
e0	6f	78	19	9a	47	74	a7	c1	23	53	89	fb	14	5d	f8	97
f0	2e	4b	b9	60	0f	ed	3e	e5	f6	87	a5	17	3a	a3	3c	b7

Table 2: Galois field logarithm table

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	01	02	04	08	10	20	40	80	87	89	95	ad	dd	3d	7a	f4
10	6f	de	3b	76	ec	5f	be	fb	71	e2	43	86	8b	91	a5	cd
20	1d	3a	74	e8	57	ae	db	31	62	c4	0f	1e	3c	78	f0	67
30	ce	1b	36	6c	d8	37	6e	dc	3f	7e	fc	7f	fe	7b	f6	6b
40	d6	2b	56	ac	df	39	72	e4	4f	9e	bb	f1	65	ca	13	26
50	4c	98	b7	e9	55	aa	d3	21	42	84	8f	99	b5	ed	5d	ba
60	f3	61	c2	03	06	0c	18	30	60	c0	07	0e	1c	38	70	e0
70	47	8e	9b	b1	e5	4d	9a	b3	e1	45	8a	93	a1	c5	0d	1a
80	34	68	d0	27	4e	9c	bf	f9	75	ea	53	a6	cb	11	22	44
90	88	97	a9	d5	2d	5a	b4	ef	59	b2	e3	41	82	83	81	85
a0	8d	9d	bd	fd	7d	fa	73	e6	4b	96	ab	d1	25	4a	94	af
b0	d9	35	6a	d4	2f	5e	bc	ff	79	f2	63	c6	0b	16	2c	58
c0	b0	e7	49	92	a3	c1	05	0a	14	28	50	a0	c7	09	12	24
d0	48	90	a7	c9	15	2a	54	a8	d7	29	52	a4	cf	19	32	64
e0	c8	17	2e	5c	b8	f7	69	d2	23	46	8c	9f	b9	f5	6d	da
f0	33	66	cc	1f	3e	7c	f8	77	ee	5b	b6	eb	51	a2	c3	00

Table 3: Galois field anti-logarithm table